An optimal O(n squared log n) bound for distinct quartics in a two-dimensional string

Panagiotis Charalampopoulos, Pawel Gawrychowski, Samah Ghazawi

arxiv:2403.06667v1; CC BY 4.0

Editorial note. Only the main distinct-quartic bound counts. Complete author proof, necessary two-dimensional periodicity context, diagrams, algorithm section and references retained; publisher author metadata supplied once by the wrapper and bibliography data mechanically expanded.

Allowed standard prerequisites (editorial).

- Standard algorithmic and mathematical background cited by the authors. All source-local constructions, proofs, appendices and references are retained; no new mathematical assumptions added.

Abstract

A fundamental concept related to strings is that of repetitions. It has been extensively studied in many versions, from both purely combinatorial and algorithmic angles. One of the most basic questions is how many distinct squares, i.e., distinct strings of the form UU, a string of length n can contain as fragments. It turns out that this is always $\mathcal{O}(n)$, and the bound cannot be improved to sublinear in n [Fraenkel and Simpson, JCTA 1998].

Several similar questions about repetitions in strings have been considered, and by now we seem to have a good understanding of their repetitive structure. For higher-dimensional strings, the basic concept of periodicity has been successfully extended and applied to design efficient algorithms—it is inherently more complex than for regular strings. Extending the notion of repetitions and understanding the repetitive structure of higher-dimensional strings is however far from complete.

Quartics were introduced by Apostolico and Brimkov [TCS 2000] as analogues of squares in two dimensions. Charalampopoulos, Radoszewski, Rytter, Waleń, and Zuba [ESA 2020] proved that the number of distinct quartics in an $n \times n$ 2D string is $\mathcal{O}(n^2 \log^2 n)$ and that they can be computed in $\mathcal{O}(n^2 \log^2 n)$ time. Gawrychowski, Ghazawi, and Landau [SPIRE 2021] constructed an infinite family of $n \times n$ 2D strings with $\Omega(n^2 \log n)$ distinct quartics. This brings the challenge of determining asymptotically tight bounds. Here, we settle both the combinatorial and the algorithmic aspects of this question: the number of distinct quartics in an $n \times n$ 2D string is $\mathcal{O}(n^2 \log n)$ and they can be computed in the worst-case optimal $\mathcal{O}(n^2 \log n)$ time.

As expected, our solution heavily exploits the periodic structure implied by occurrences of quartics. However, the two-dimensional nature of the problem introduces some technical challenges. Somewhat surprisingly, we overcome the final challenge for the combinatorial bound using a result of Marcus and Tardos [JCTA 2004] for permutation avoidance on matrices.

1 Introduction

Repetitions are a staple topic of both combinatorics on words [21] and algorithms on strings [32]. In both areas, the classical objects of study are linear sequences of characters from a finite alphabet. Depending on whether we are more interested in their combinatorial properties or designing efficient algorithms for them, it is customary to call such sequences words or strings, respectively. In this paper, we use the latter convention.

Perhaps the most natural example of a repetition in a string is a square, that is, a string of the form UU, also known as a "tandem repeat" in the biological literature [53]. The basic question concerning squares is whether any of the fragments of a string of length n is a square, and, if so, what is the number of such fragments. The origins of this question can be traced back to Thue [75], who constructed an infinite string over a ternary alphabet that contains no squares. Thus, we can construct arbitrarily long square-free strings over such alphabets. The next question is what is the largest possible number of fragments that are squares. However, any even-length fragment of $\mathtt{a}^n$ is a square. One way to make the question non-trivial is to only consider the primitively rooted squares, meaning that U is not a power of another string. This decreases the possible number of occurrences to $\mathcal{O}(n \log n)$, which is asymptotically tight [28]. Another way is to only consider distinct squares.

Fraenkel and Simpson [46] showed that any string of length n contains at most $2n$ distinct squares and constructed an infinite family of strings of length n containing $n - \Theta(n)$ distinct squares. For many years, it was conjectured that the upper bound should be at most n. After a series of simplifications and improvements [40, 57, 58, 66, 74], the conjecture was finally proven by Brlek and Li [23], who showed an upper bound of $n - \sigma + 1$, where σ is the size of the alphabet. The same authors [24] also showed an upper bound of $n - \Theta(\log n)$. On the algorithmic side, Apostolico and Preparata [16], Main and Lorentz [68] and Crochemore [29] showed how to find a compact representation of all squares (in particular, test square-freeness) in a string of length n in $\mathcal{O}(n \log n)$ time. Specifically, such a representation stores all distinct squares. To obtain a faster algorithm for finding only the distinct squares, one needs to restrict the size of the alphabet. For constant alphabets, Gusfield and Stoye [54] designed an $\mathcal{O}(n)$ time algorithm. This was later generalized to the more general case of an integer alphabet (that can be sorted in linear time) [19,35]. The complexity of testing square-freeness over general ordered and unordered alphabets was very recently settled by Ellert and Fischer [42] and Ellert et al. [43], respectively; this problem has been also studied in the parallel [13, 14, 36, 37] and the online settings [55, 64, 65, 67]. Thus, by now we seem to have obtained a rather good understanding of both the combinatorial and the algorithmic properties of distinct squares.

Arguably, linear sequences are not always best suited to model the objects that we would like to study. A natural extension is to consider rectangular arrays of characters from a finite alphabet, which can be seen as 2D strings. Possible applications in image processing [72] sparked interest in designing algorithms for searching in 2D strings already in the late 1970s [17, 22]. This turned out to be significantly more challenging than searching in 1D strings: both versions were studied already in the 70s, but while for 1D strings an alphabet-independent linear-time algorithm had been soon found [60], achieving the same goal for 2D strings took till the 90s [3, 38, 47]. Extensions of this basic problem such as approximate searching [9,27], indexing [26,49,50], searching in smaller space [31], scaled searching [6,7], searching in random 2D strings [59], dictionary searching [8,56,70], and searching in compressed 2D strings [1, 4, 11] have been also considered.

The combinatorial structure of 2D strings seems to be significantly more involved than that of 1D strings. As a prime example, the basic tool used in algorithms and combinatorics on 1D strings is periodicity. We say that p is a period of a 1D string $S[1\,..\,n]$ when $S[i] = S[i + p]$ for all

$i = 1, 2, \ldots, n - p$. The set of all periods is very structured due to a classical result of Fine and Wilf [45] according to which for any two periods p, q such that $p + q \leq n$, the greatest common divisor of p and q is also a period. The natural way to extend this notion to 2D strings is to define (x, y) to be a period of a 2D string $S[1\,..\,n][1\,..\,n]$ when $S[i + x][j + y]$ for all $i = 1, 2, \ldots, n - x$ and $j = 1, 2, \ldots, n - y$. This notion was introduced by Amir and Benson [2], who provided a detailed study based on classifying 2D strings into four periodicity classes. This classification was later crucial in designing both an alphabet-independent linear-time algorithm [38,47] and alphabet-independent optimal parallel algorithms [5,30].

The rich combinatorial structure of 2D strings brings the challenge of finding the right generalization of the concept of repetitions. Apostolico and Brimkov [12] introduced two notions of repetitions in 2D strings that can be seen as natural analogues of squares in 1D strings. A tandem $W^{1,2}$ (or $W^{2,1}$) consists of 2 occurrences of the same block W arranged in a 1×2 (or 2×1) pattern. Next, a quartic $W^{2,2}$ consists of 4 occurrences of the same block W arranged in a 2×2 pattern. Note that Apostolico and Brimkov [12] additionally required that W is primitive, meaning that it cannot be partitioned into non-overlapping copies of another block. However, it is more natural to call such tandems and quartics primitively rooted, as in [25]. Apostolico and Brimkov [12] showed asymptotically tight bounds of $\mathcal{O}(n^3 \log n)$ and $\mathcal{O}(n^2 \log^2 n)$ for the number of primitively rooted tandems and quartics, respectively. The former bound was later complemented with a worst-case optimal $\mathcal{O}(n^3 \log n)$-time algorithm [15].

Two tandems $T = W^{1,2}$ and $T' = V^{1,2}$ are distinct when $W \neq V$. Similarly, two quartics $Q = W^{2,2}$ and $Q' = V^{2,2}$ are distinct when $W \neq V$. It is easy to see that an $n \times n$ 2D string contains $\mathcal{O}(n^3)$ distinct tandems by applying the bound on the number of 1D distinct squares on every horizontal slice of the 2D string. It is also not hard to show that this bound is asymptotically tight, even over a binary alphabet [48]. Thus, tandems do not seem to be the right generalization of squares, and we should rather focus on quartics.

Recently, Charalampopoulos, Radoszewski, Rytter, Waleń, and Zuba [25] showed a non-trivial upper bound of $\mathcal{O}(n^2 \log^2 n)$ on the number of distinct quartics in an $n \times n$ 2D string, and an algorithm that finds them in the same time complexity. At this point, it was quite unclear to what extent distinct quartics suffer from the "curse of dimensionality". Could it be that, similarly to the number of distinct squares, their number is also linear in the size of the input? Gawrychowski, Ghazawi, and Landau [48] very recently showed that this is not the case, by constructing an infinite family of $n \times n$ 2D strings over a binary alphabet containing $\Omega(n^2 \log n)$ distinct quartics. This shows that there is a qualitative difference between distinct squares and distinct quartics, but leaves a significant gap between the lower bound of $\Omega(n^2 \log n)$ and the upper bound of $\mathcal{O}(n^2 \log^2 n)$ [25].

Our Results. Our contribution is twofold. First, we show an asymptotically tight bound of $\mathcal{O}(n^2 \log n)$ on the number of distinct quartics in an $n \times n$ 2D string. Thus, the "curse of dimensionality" for this problem is a single logarithm for going from 1D to 2D. Second, we show how to find all distinct quartics in worst-case optimal $\mathcal{O}(n^2 \log n)$ time. We thus resolve both the combinatorial and algorithmic complexity of distinct quartics.

A notable difference of our algorithm from the previously fastest algorithm for computing distinct quartics [25] is that the algorithm of [25] first finds all 2D runs[1] of the 2D string, which are not even known to be $\mathcal{O}(n^2 \log n)$, and then infers the quartics from those. We manage to circumvent this, by focusing on some selected occurrences of 2D strings of the form $Q^{5,5}$, instead of considering all of them via 2D runs.

[1] 2D runs are subarrays that are periodic both vertically and horizontally and cannot be extended without any of the periods changing.

Overview of the Combinatorial Upper Bound. When bounding the number of distinct squares, one begins with fixing the rightmost occurrence of every distinct square [46]. In two dimensions, it is less clear what an extreme occurrence could mean. We simply say that it is an occurrence at a position (i, j) such that there is no other occurrence at a different position (i', j') such that $i' \geq i$ and $j' \geq j$. Next, a standard trick used when working with strings is to partition them into groups with length in $[2^a \,..\, 2^{a+1})$ for different integers a. Similarly to previous work [25], we partition quartics into groups $C_{(a,b)}$ with height in $[2^a \,..\, 2^{a+1})$ and width in $[2^b \,..\, 2^{b+1})$ for pairs of integers (a, b). We begin with proving that, for any position (i, j), the set of extreme occurrences at (i, j) may have a non-empty intersection with only $\mathcal{O}(\log n)$ such groups. Next, we partition all quartics into thin and thick (note that the meaning of thin and thick is slightly different than in the previous work [25]). More specifically, a quartic Q is thick if and only if it can be partitioned into $x \times y$ occurrences of a primitive 2D string R, i.e., $Q = R^{x,y}$ for some $x, y \geq 5$. Then, we show that for any position (i, j) and group $C_{(a,b)}$, there can be at most 10 extreme occurrences of thin quartics in $C_{(a,b)}$ at position (i, j). Overall, we thus have only $\mathcal{O}(n^2 \log n)$ distinct thin quartics.

The main part of our proof for the combinatorial upper bound is the analysis of the number of extreme occurrences of thick quartics. Our starting point is the observation (already present in [25]) that this number can be upper bounded by the number of occurrences of 2D strings of the form $R^{5,5}$, for primitive R, that participate in the partition of an extreme occurrence of some quartic $R^{x,y}$. To bound the number of such occurrences, we assign an occurrence of $R^{5,5}$ at position (i, j) to position $(x, y) = (i + 2 \cdot \mathsf{height}(R), j + 2 \cdot \mathsf{width}(R))$ and say that this occurrence is anchored at position (x, y). Then, our goal is to show that the number of occurrences assigned to every position is only $\mathcal{O}(\log n)$. For a fixed position (i, j), this is done by first arguing that the pairs $(\lfloor \log(\mathsf{height}(R)) \rfloor, \lfloor \log(\mathsf{width}(R)) \rfloor)$ are pairwise distinct among occurrences of different $R^{5,5}$ assigned to (i, j). This requires a careful analysis of the implied periodic structure and allows us to focus on bounding the number of such pairs. What we do next is the main novelty of our approach for the combinatorial upper bound. We treat the pairs as a set of points $\mathcal{P} \subseteq [1 \,..\, m]^2$, where $m = \lfloor \log n \rfloor$, and argue that, for each $(a, b) \in \mathcal{P}$, the set of points of $\mathcal{P}$ that are strictly dominated by (a, b) can be partitioned into at most two chains. Next, our goal is to upper bound the size of any set $\mathcal{P}$ with this property by $\mathcal{O}(m)$. To this end, we leverage a result from extremal combinatorics, namely, the proof of the Füredi-Hajnal conjecture by Marcus and Tardos [69]. This result states that, if an $m \times m$ binary matrix M avoids a fixed permutation matrix P as a submatrix, i.e., if P cannot be obtained by deleting some rows and columns of M and changing 1s to 0s, then it contains at most $c_P \cdot m$ 1s, where c_P is a constant if the size of P is a constant. We reformulate the constraint on $\mathcal{P}$ to avoid the permutation matrix shown below as a submatrix. Overall, this allows us to conclude that the number of extreme occurrences of thick quartics is also $\mathcal{O}(n^2 \log n)$.

		1	
	1		
1			
			1

A high-level description of our approach for the algorithmic part is provided in Section 4.1, as it is best read after the full proof of the combinatorial upper bound.

Open Problem. An interesting follow-up question on repetitions in 2D strings is that of settling the number of 2D runs that a 2D string can have. Charalampopoulos et al. [25] proved an $\mathcal{O}(n^2 \log^2 n)$ upper bound for the number of 2D runs that an $n \times n$ 2D string can contain, while Gawrychowski et al. [48] constructed an infinite family of $n \times n$ 2D strings (over a binary alphabet) with $\Omega(n^2 \log n)$

2D runs. On the algorithms' side, Amir et al. [10] devised an algorithm that computes all 2D runs in an $n \times n$ 2D string in $\mathcal{O}(n^2 \log n + |\mathsf{output}|)$ time, and is thus optimal. For 1D strings, after a long line of results [33,34,51,52,63,71,73] the number of runs was shown to be less than n [18] and they can be computed in $\mathcal{O}(n)$ time for strings over ordered alphabets [42] (see [18,63] for earlier algorithms for strings over linear-time sortable alphabets).

2 Preliminaries

For integers $i \leq j$, we denote the integer interval $\{i, \ldots, j\}$ by either of $[i\,..\,j]$, $(i-1\,..\,j+1)$, $[i\,..\,j+1)$, and $(i-1\,..\,j]$.

Let us consider a string $S = S[1]S[2]\cdots S[n]$ of length $|S| = n$. For integers $i \leq j$ in $[1\,..\,n]$, we denote the fragment $S[i]\cdots S[j]$ by $S[i\,..\,j]$. A non-trivial rotation of a string S of length n is a string $S[j\,..\,n]S[1\,..\,j]$ for $j \in [1\,..\,n)$. A positive integer $p \leq n$ is a period of S if and only if $S[i] = S[i+p]$ for all $i \in [1\,..\,n-p]$. The smallest period of S is called the period of S and is denoted by $\mathsf{per}(S)$. A string is called periodic if and only if its period is at most half its length. We will extensively use the following property of periods.

Lemma 2.1 (Periodicity Lemma [45]). If p and q are periods of a string S and satisfy $p + q \leq |S|$, then $\gcd(p, q)$ is also a period of S.

We denote the concatenation of two strings U and V by UV. Further, for $k \in \mathbb{Z}_+$, we denote the concatenation of k copies of U by U^k. A string V that cannot be written as U^k for a string U and an integer $k > 1$ is called primitive. A string of the form UU is called a square. A square UU is said to be primitively rooted if U is primitive. More generally, a string of the form U^k is called a k-th power, and it is said to be primitively rooted if U is primitive. We extensively use the following property of squares.

Lemma 2.2 (Three Squares Lemma [39][2]). If squares U^2 and V^2 are proper prefixes of a square W^2, $|U| < |V|$, and U is primitive, then $|U| + |V| \leq |W|$.

We next summarise some combinatorial properties of squares and higher powers.

Proposition 2.3. Consider a string S and an integer a. At most two prefixes of S with lengths from $[2^a\,..\,2^{a+1})$ can be primitively rooted squares.

Proof. Assume that there are three such prefixes, and denote them by UU, VV, WW, where $|U| < |V| < |W|$. Since $|UU|, |VV|, |WW| \in [2^a\,..\,2^{a+1})$, we have $|U| + |V| > |W|$, which together with the primitivity of $|U|$ leads to a contradiction. □

Proposition 2.4. Consider a string S and an integer a. All prefixes of S with lengths from $[2^a\,..\,2^{a+1})$ that are powers higher than 2 are of the form U^k for the same primitive string U.

Proof. Assume that there are two such prefixes U^k and V^ℓ, where U is primitive, $k, \ell \geq 3$ and $|U| < |V|$. Since $|U^k|, |V^\ell| \in [2^a\,..\,2^{a+1})$ and $k, \ell \geq 3$, we have $|U| + |V| \leq |U^k|$. Next, $|U|$ and $|V|$ are periods of U^k, so by Lemma 2.1 we obtain that $\gcd(|U|, |V|)$ is a period of U^k. But U is primitive, so $\gcd(|U|, |V|) = |U|$, and V is a power of U. By repeating this reasoning, we obtain that all such prefixes are powers of the same primitive U. □

[2]This formulation comes from [46].

W	W	W	W	W
W	W	W	W	W

Figure 1: 2D string $W^{2,5}$ is shown for some 2D string W.

A fragment $S[i\,..\,j]$ of a string S is a run if and only if it is periodic and it cannot be extended by a character in either direction with its period remaining unchanged.

An $m \times n$ 2D string A is simply a two-dimensional array with m rows and n columns, where $\mathsf{height}(A) = m$ and $\mathsf{width}(A) = n$. The position that lies on the i-th row and the j-th column of A is position (i, j). We say that a 2D string P occurs at a position (i, j) of a 2D string T if and only if the subarray (also called fragment) $T[i\,..\,i+\mathsf{height}(P))[j\,..\,j+\mathsf{width}(P))$ of T equals P. We write $\Sigma^{*,*}$ to denote the set of all 2D strings over alphabet Σ.

A positive integer p is a horizontal period of a 2D string A such that $\mathsf{width}(A) \geq p$ if and only if the j-th column of A is equal to the $(j+p)$-th column of A for all $j \in [1\,..\,\mathsf{width}(A) - p]$. The smallest horizontal period of A is the horizontal period of A. An integer q is a (the) vertical period of A if and only if q is a (resp. the) horizontal period of the transpose of A.

It will be sometimes convenient to view a 2D string as a 1D metastring by viewing each column (or row) as a metacharacter such that metacharacters are equal if and only if the corresponding columns (resp. rows) are equal. Observe that the horizontal periods (resp. vertical periods) of a 2D string A are in one-to-one correspondence with the periods of the metastring obtained from A by viewing each column (resp. row) as a metacharacter.

For a 2D string W and $x, y \in \mathbb{Z}_+$, we denote by $W^{x,y}$ the 2D string that consists of $x \times y$ copies of W; see Figure 1 for an illustration. A 2D string W is primitive if it cannot be written as $Y^{a,b}$ for any 2D string Y and $a, b \in \mathbb{Z}_+$ that are not both equal to 1. The primitive root of a 2D string X is the unique primitive 2D string Y such that $X = Y^{a,b}$ for $a, b \in \mathbb{Z}_+$. Note that the primitive root is indeed unique by the periodicity lemma applied to the horizontal and vertical 1D metastrings obtained from X.

Model of computation. For our algorithm, we assume the standard word-RAM model of computation with word-size $\Omega(\log n)$.

3 The Combinatorial Bound

We consider an $n \times n$ 2D string A, whose entries are over an arbitrary alphabet Σ. We say that a fragment $A[i\,..\,i')[j\,..\,j')$ is a quartic-fragment if and only if it equals some quartic Q; further, we say that it is an extreme or bottom-right quartic-fragment if Q does not have any occurrence at another position (i'', j'') with $i'' \geq i$ and $j'' \geq j$. We refer to such an occurrence of Q as an extreme or bottom-right occurrence. We denote by $\mathsf{BR}(i, j)$ the set of extreme quartic-fragments with top-left corner (i, j). Further, we denote the union of all $\mathsf{BR}(i, j)$ by BR. Observe that the distinct quartics in BR are exactly the distinct quartics in A as every quartic that occurs in A has at least one extreme occurrence. Note that a quartic may have $\Theta(n)$ extreme occurrences; an example is provided in Figure 2.

Let us consider a partition of the quartic-fragments of A into $\mathcal{O}(\log^2 n)$ canonical sets, such that, for each $(a, b) \in [1\,..\,\lfloor \log n \rfloor]^2$, the canonical set $C_{(a,b)}$ consists of all quartic-fragments of A whose height is in $[2^a\,..\,2^{a+1})$ and whose width is in $[2^b\,..\,2^{b+1})$.[3]

[3]Throughout this work, logarithms have base 2.

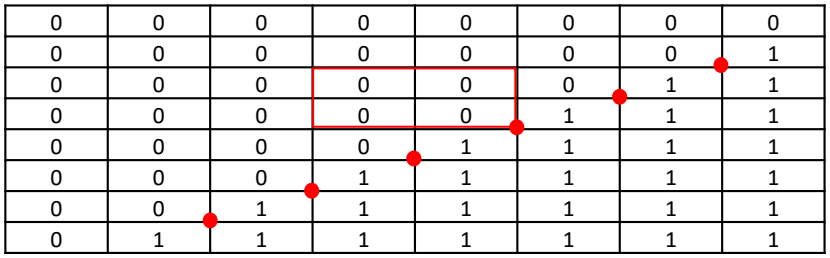


Figure 2: Consider an $n \times n$ 2D string A all of whose entries that lie weakly above the main diagonal are equal to 0 and all of whose entries that lie strictly below the main diagonal are equal to 1. The quartic that equals $0^{2,2}$ has $n-2$ extreme occurrences in A. This is illustrated for $n = 8$: the bottom-right corners of extreme occurrences of said quartic are marked.

Lemma 3.1. For each position (i, j) of A, $\mathit{BR}(i, j)$ has a non-empty intersection with $\mathcal{O}(\log n)$ canonical sets.

Proof. We say that the aspect ratio of a quartic Q is equal to 2 raised to the power $\lfloor \log(\mathsf{height}(Q)) \rfloor - \lfloor \log(\mathsf{width}(Q)) \rfloor$. The aspect ratio of all quartic-fragments in a canonical set $C_{(a,b)}$ is 2^{a-b}. Observe, that there are $2 \cdot \lfloor \log n \rfloor - 1$ different possible values for the aspect ratio of a quartic. For each $d \in [-\lfloor \log n \rfloor + 1 \,..\, \lfloor \log n \rfloor - 1]$, let $\mathsf{BR}_d(i, j)$ be the subset of $\mathsf{BR}(i, j)$ that contains exactly the elements of $\mathsf{BR}(i, j)$ with aspect ratio 2^d.

Next, we show that, for each d, we have at most two canonical sets contributing to $\mathsf{BR}_d(i, j)$. Let us suppose towards a contradiction that we have three canonical sets $C_{(a,b)}$, $C_{(a',b')}$, and $C_{(a'',b'')}$ that contribute to $\mathsf{BR}_d(i, j)$. In other words, there are quartic-fragments

- $Q \in \mathsf{BR}_d(i, j)$ with height in $[2^a \,..\, 2^{a+1})$ and width in $[2^b \,..\, 2^{b+1})$,
- $Q' \in \mathsf{BR}_d(i, j)$ with height in $[2^{a'} \,..\, 2^{a'+1})$ and width in $[2^{b'} \,..\, 2^{b'+1})$, and
- $Q'' \in \mathsf{BR}_d(i, j)$ with height in $[2^{a''} \,..\, 2^{a''+1})$ and width in $[2^{b''} \,..\, 2^{b''+1})$.

Since $a - b = a' - b' = a'' - b'' = d$, we can assume without loss of generality that $a < a' < a''$ and $b < b' < b''$. We thus have that $a+1 < a''$ and $b+1 < b''$, which implies that Q is fully contained in the top left quarter of Q''. Thus, Q has an occurrence at position $(i+\mathsf{height}(Q'')/2, j+\mathsf{width}(Q'')/2)$; see Figure 3. This contradicts our assumption that the occurrence of Q at position (i, j) is an extreme occurrence.

Thus, $\mathcal{O}(\log n)$ canonical sets contribute to $\mathsf{BR}(i, j)$: at most two for each aspect ratio. □

Henceforth, we call a quartic Q with primitive root P thick if $Q = P^{x,y}$ for $x, y \geq 5$ and thin otherwise.

Lemma 3.2. For any position (i, j) of A and any pair $(a, b) \in [1 \,..\, \lfloor \log n \rfloor]^2$, $C_{(a,b)} \cap \mathit{BR}(i, j)$ can contain at most 10 thin quartics.

Proof. The possible forms of thin quartics are $P^{2,2}$, $P^{2,2x}$, $P^{2y,2}$, $P^{4,2x}$, and $P^{2y,4}$ for $x > 1$ and $y > 1$. We will consider each form separately.

First, we consider quartics of the form $P^{2,2}$ in $C_{(a,b)} \cap \mathsf{BR}(i, j)$. We analyse the fragment $A[i \,..\, n][j \,..\, j + 2^b)$ and treat it as a metastring by viewing rows as metacharacters. We observe that each considered quartic defines a prefix that is a square there. Further, all those squares need

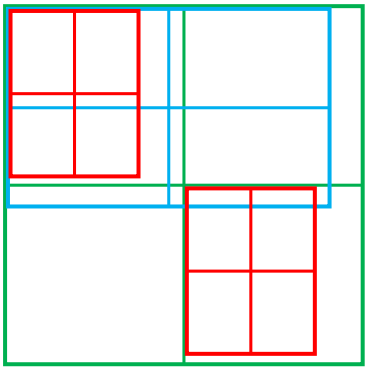


Figure 3: An illustration of the proof of Theorem 3.1 with quartics Q, Q', and Q'' drawn in red, blue, and green, respectively.

to be primitive, as otherwise P could be written as $P = Q^{k,1}$, for some $k > 1$, contradicting the primitivity of P. Thus, by Theorem 2.3 we have at most two possible heights for the considered quartics. By a symmetric argument, we have at most two possible widths, and so at most 4 quartics.

Second, we consider quartics of the form $P^{2,2x}$ in $C_{(a,b)} \cap \mathsf{BR}(i,j)$. By the same reasoning as above, we have at most two possible heights for the considered quartics; let h by one of them. We analyse the fragment $A[i\,..\,i+h)[j\,..\,n]$ and treat it as a metastring by viewing columns as metacharacters. Each considered quartic with height h corresponds to a prefix that is a $(2x)$-th power, for some $x > 1$. By Theorem 2.4, all such prefixes are powers of the same U; let U^{2x} be the longest such prefix. Then, for any $x' < x$, the prefix $U^{2x'}$ also occurs at position $(i, j + |U|)$, so the occurrence at position (i,j) cannot be an extreme occurrence. Therefore, for every possible height, we have at most one quartic, so at most 2 in total.

Third, we consider quartics of the form $P^{4,2x}$ for $x > 1$ in $C_{(a,b)} \cap \mathsf{BR}(i,j)$. We (again) analyse the fragment $A[i\,..\,n][j\,..\,j+2^b)$ and treat it as a metastring by viewing its rows as metacharacters. We observe that each considered quartic defines a prefix that is a primitively rooted fourth power there. Thus, by Theorem 2.4 we have at most one possible height, and by the same reasoning as above at most one quartic.

Symmetric arguments bound the number of quartics of the forms $P^{2y,2}$ and $P^{2y,4}$. □

Lemma 3.3. The number of distinct thin quartics in A is $\mathcal{O}(n^2 \log n)$.

Proof. For each position (i,j) of A, $\mathsf{BR}(i,j)$ has a non-empty intersection with at most $\mathcal{O}(\log n)$ canonical sets due to Theorem 3.1. Further, by Theorem 3.2, there are at most 10 thin quartics in each such intersection. Since A has n^2 positions, the stated bound follows. □

3.1 Reduction to a Geometric Problem

We next partition the thick elements of BR by primitive root. For each primitive 2D string R, let us denote the thick elements of BR with primitive root R by BR_R. Additionally, let us denote by $\mathsf{occ}_{5\times5}(R)$ the set of all positions (i,j) of A where $R^{5,5}$ occurs such that there is an element of BR_R that fully contains this occurrence of $R^{5,5}$ and has top-left corner equal to $(i - x \cdot \mathsf{height}(R), j - y \cdot \mathsf{width}(R))$ for some non-negative integers x and y.

The proof of the following lemma proceeds almost exactly as the proof of Claim 18 in [25], except that we work with occurrences of $R^{5,5}$ instead of $R^{3,3}$ and do not need the notion of special points. We provide a detailed description for completeness.

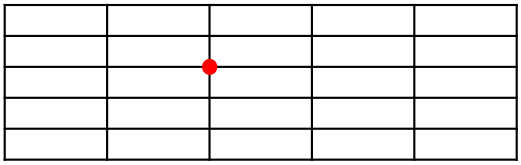


Figure 4: The red point corresponds to the anchor of the shown occurrence of $R^{5,5}$.

Lemma 3.4 (cf. the proof of [25, Claim 18]). For any 2D string R, $|\mathit{BR}_R| \leq |\mathit{occ}_{5\times 5}(R)|$.

Proof. We will map each $Q \in \mathsf{BR}_R$ to an occurrence of $R^{5,5}$ in such a way that two distinct quartic-fragments $Q, Q' \in \mathsf{BR}_R$ are mapped to distinct occurrences. This will imply that the number of occurrences of R is at least as large as the number of elements of BR_R.

For each $x = 6, 8, \ldots$ in this order, we select $Q \in \mathsf{BR}_R$ such that $\mathsf{height}(Q) = x \cdot \mathsf{height}(R)$ and $\mathsf{width}(Q) = y \cdot \mathsf{width}(R)$ is the largest among all $Q' \in \mathsf{BR}_R$ with $\mathsf{height}(Q') = x \cdot \mathsf{height}(R)$. We note that the number of $Q' \in \mathsf{BR}_R$ with $\mathsf{height}(Q') = x \cdot \mathsf{height}(R)$ is at most $y/2 - 2$, and our goal is to map them to occurrences of $R^{5,5}$ that have not been used so far. Additionally, we will ensure that those occurrences are all in the same row. Let (i, j) be the position of an extreme occurrence of Q. We observe that $R^{5,5}$ occurs at every position (i', j') with $i' = i + k \cdot \mathsf{height}(R)$ and $j' = j + \ell \cdot \mathsf{width}(R)$, for every $k \in [0\,..\,x-5]$ and $\ell \in [0\,..\,y-5]$. We choose $k \in [0\,..\,x-5]$ such that none of the occurrences of $R^{5,5}$ at positions $(i + k \cdot \mathsf{height}(R), j + \ell \cdot \mathsf{width}(R))$, for $\ell = 0, 1, \ldots, y-3$ have been used so far. This is possible because so far we have used occurrences of $R^{5,5}$ in only $x/2 - 3 < x - 4$ rows. Then, we map every $Q' \in \mathsf{BR}_R$ with $\mathsf{height}(Q') = x \cdot \mathsf{height}(R)$ to an occurrence of $R^{5,5}$ at position $(i + k \cdot \mathsf{height}(R), j + \ell \cdot \mathsf{width}(R))$, for some $\ell \in [0\,..\,y-5]$, which is possible due to $y/2 - 2 \leq y - 4$. □

Thus, it remains to upper bound $\sum_R |\mathsf{occ}_{5\times 5}(R)|$, i.e., the sum, over all R, of the number of occurrences of $R^{5,5}$ which are contained in some element of BR_R.

Consider an occurrence of a 2D string of the form $R^{5,5}$, for a primitive string R, at a position (i, j) of A. We call position $(i + 2 \cdot \mathsf{height}(R), j + 2 \cdot \mathsf{width}(R))$ the anchor of this occurrence; see Figure 4.

Now, for each primitive string R, for each element of $\mathsf{occ}_{5\times 5}(R)$, we assign the corresponding occurrence of $R^{5,5}$ to its anchor. Let $\mathsf{assign}(i, j)$ be the set of primitive 2D strings R such that occurrences of $R^{5,5}$ have been assigned to position (i, j). We have

$$\sum_R |\mathsf{occ}_{5\times 5}(R)| = \sum_{i=1}^{n} \sum_{j=1}^{n} |\mathsf{assign}(i, j)|. \tag{1}$$

It now suffices to show that $\sum_{i=1}^{n} \sum_{j=1}^{n} |\mathsf{assign}(i, j)| = \mathcal{O}(n^2 \log n)$. We will in fact show that $|\mathsf{assign}(i, j)| = \mathcal{O}(\log n)$ for all i, j, which straightforwardly yields the desired bound.

Let us fix a position (i, j). By applying Theorem 2.4 horizontally and vertically one easily obtains the following fact.

Fact 3.5 ([25, Corollary 13]). Let a, b be non-negative integers and W, Z be different 2D strings with height in $[2^a\,..\,2^{a+1})$ and width in $[2^b\,..\,2^{b+1})$. If $W^{3,3}$ and $Z^{3,3}$ occur at some position of A, then at least one of W and Z is not primitive.

This, together with the fact that, for each $R \in \mathsf{assign}(i, j)$, $R^{3,3}$ occurs at position (i, j), implies the following.

Fact 3.6. For each pair $(a, b) \in [1 \,..\, \lfloor \log n \rfloor]^2$, for each position (i, j) of A, the set $\mathit{assign}(i, j)$ contains at most one element with height in $[2^a \,..\, 2^{a+1})$ and width in $[2^b \,..\, 2^{b+1})$.

Let us define a map $\Sigma^{*,*} \to [1 \,..\, \lfloor \log n \rfloor]^2$ as $g(R) \mapsto (\lfloor \log(\mathsf{height}(R)) \rfloor, \lfloor \log(\mathsf{width}(R)) \rfloor)$. Let f be the restriction of g to the domain $\mathsf{assign}(i, j)$. Due to Theorem 3.6, f is an injective function. We henceforth identify each element R of $\mathsf{assign}(i, j)$ with point $f(R)$. We denote the image of f by $\mathcal{P}$.

We say that a point $(a, b) \in \mathbb{Z}^2$ dominates a point (a', b') if and only if $a' \leq a$ and $b' \leq b$; the dominance is said to be strict if and only if $a' < a$ and $b' < b$; the dominance is said to be strong if and only if $a' \leq a$ and $b' < b$ or $a' < a$ and $b' \leq b$; the dominance is said to be weak if and only if $a' \leq a$ and $b' \leq b$. A set of points on which the domination relation forms a total order is called a chain. A set of points such that none dominates another is called an antichain. We are going to use Dilworth's theorem [41], which states that, in any finite partially ordered set, the size of the largest antichain is equal to the minimum number of chains in which the elements of the set can be decomposed.

For two primitive 2D strings S and T, with $3 \cdot \mathsf{height}(S) \leq 2 \cdot \mathsf{height}(T)$ and $\mathsf{width}(S) < \mathsf{width}(T)$, we say that S horizontally spans T when the 2D string $\mathsf{row}_{3 \cdot \mathsf{height}(S)}(T^{2,2})$, consisting of the $3 \cdot \mathsf{height}(S)$ topmost rows of $T^{2,2}$, equals $S^{3,y}$ for some integer $y \geq 2$. Similarly, when $3 \cdot \mathsf{width}(S) \leq 2 \cdot \mathsf{width}(T)$ and $\mathsf{height}(S) < \mathsf{height}(T)$, we say that S vertically spans T when the 2D string $\mathsf{col}_{3 \cdot \mathsf{width}(S)}(T^{2,2})$, consisting of the $3 \cdot \mathsf{width}(S)$ leftmost columns of $T^{2,2}$, equals $S^{x,3}$ for some integer $x \geq 2$.

Fact 3.7. Let S and T be two primitive 2D strings. If S spans T horizontally, then the horizontal period of $\mathit{row}_{3 \cdot \mathit{height}(S)}(T^{2,2})$ is $\mathit{width}(S)$. Symmetrically, if S spans T vertically, then the vertical period of $\mathit{col}_{3 \cdot \mathit{width}(S)}(T^{2,2})$ is $\mathit{height}(S)$.

Proof. We only prove the first statement as the second one follows by symmetry. Let us view $\mathsf{row}_{3 \cdot \mathsf{height}(S)}(T^{2,2})$ as a metastring Z by viewing each of its columns as a metacharacter; the horizontal period of $\mathsf{row}_{3 \cdot \mathsf{height}(S)}(T^{2,2})$ equals $p := \mathsf{per}(Z)$. Note that $\mathsf{width}(S)$ is a period of Z. Towards a contradiction, suppose that $p < \mathsf{width}(S)$. Then, an application of the periodicity lemma to Z implies that p must divide $\mathsf{width}(S)$. This fact contradicts the primitivity of S, as we would have that $S = (S[1 \,..\, \mathsf{height}(S)][1 \,..\, p])^k$ for $k = \mathsf{width}(S)/p$. □

When reading the following lemma, one can think of M being in $\mathsf{assign}(i, j)$. However, the lemma is slightly more general, as needed for the algorithm that is presented in Section 4.

Lemma 3.8. Let $R \in \mathit{assign}(i, j)$ and M be a primitive 2D string such that:

- $M^{5,5}$ has an occurrence with anchor (i, j);
- $g(M)$ strictly dominates $g(R)$.

Then, R spans M either horizontally or vertically (or both).

Proof. By the definition of $\mathsf{assign}(i, j)$, the occurrence of $R^{5,5}$ assigned to position (i, j) appears inside an element of BR_R, that is, a bottom-right occurrence of a thick quartic with primitive root R. Let us denote this quartic by Q. Observe that the considered occurrence of Q cannot be fully contained inside the occurrence of $M^{4,4}$ at position $(i - 2 \cdot \mathsf{height}(M), j - 2 \cdot \mathsf{width}(M))$ as this would contradict the fact that the considered occurrence of Q is bottom-right: there would be another occurrence $\mathsf{width}(M)$ positions to the right. Therefore, Q must contain at least one of the following four fragments of A, depicted in Fig. 5: $A[j \,..\, j + 3 \cdot \mathsf{height}(R))[i \,..\, i + 2 \cdot \mathsf{width}(M))$,

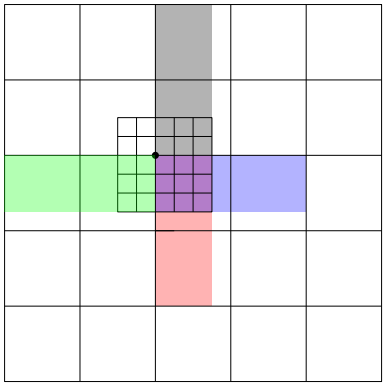


Figure 5: The considered occurrences of each of $M^{5,5}$ and $R^{5,5}$ are shown, together with the four specified fragments, at least one of which must be fully contained in Q.

$A[j\,..\,j+3\cdot\mathsf{height}(R))[i-2\cdot\mathsf{width}(M)\,..\,i)$, $A[j\,..\,j+2\cdot\mathsf{height}(M))[i\,..\,i+3\cdot\mathsf{width}(R))$, $[j-2\cdot\mathsf{height}(M)\,..\,j)A[i\,..\,i+3\cdot\mathsf{width}(R))$.

We next show that in either of the first two cases, R horizontally spans M. In the remaining cases, a symmetric argument yields that R vertically spans M. We first argue that, in either of the first two cases, we have $3\cdot\mathsf{height}(R)\leq 2\cdot\mathsf{height}(M)$. If this were not the case, $\mathsf{height}(R)$ would be a vertical period of $M^{2,2}$ since $\mathsf{height}(R)+\mathsf{height}(M)\leq 2\mathsf{height}(M)$. Then, a direct application of Lemma 2.1 to the metastring obtained from $M^{2,2}$ by viewing its rows as metacharacters, would contradict the primitivity of M. The fact that $\mathsf{width}(R)<\mathsf{width}(M)$ is a direct consequence of $g(M)$ strictly dominating $g(R)$. Then, in either of the considered cases, we have that both $\mathsf{width}(M)$ and $\mathsf{width}(R)$ are horizontal periods of $\mathsf{row}_{3\cdot\mathsf{height}(R)}(M^{2,2})$. Hence, by the periodicity lemma applied to the metastring obtained from $\mathsf{row}_{3\cdot\mathsf{height}(R)}(M^{2,2})$ by viewing each of its columns as a metacharacter, we have that $\mathsf{width}(R)$ divides $\mathsf{width}(M)$. This completes the proof. □

Lemma 3.9. Two primitive 2D strings R_1 and R_2 such that $g(R_1)$ and $g(R_2)$ form an antichain cannot both horizontally span a 2D string R.

Proof. Let $g(R_1)=(a_1,b_1)$ and $g(R_2)=(a_2,b_2)$. Without loss of generality, we can assume that $a_1<a_2$ and $b_1>b_2$. Suppose towards a contradiction that both R_1 and R_2 horizontally span R. By applying Theorem 3.7, we obtain that

- $\mathsf{width}(R_1)$ is the horizontal period of $\mathsf{row}_{3\cdot\mathsf{height}(R_1)}(R^{2,2})$ and
- $\mathsf{width}(R_2)$ is the horizontal period of $\mathsf{row}_{3\cdot\mathsf{height}(R_2)}(R^{2,2})$.

Note that, for any $k\in[1\,..\,\mathsf{height}(R)]$, the horizontal period of the string comprised of the k topmost rows of $R^{1,2}$ equals the least common multiple of the periods of those k rows. Hence, the period cannot decrease as we increase the number of considered rows. We thus have $\mathsf{width}(R_2)\leq\mathsf{width}(R_1)$ since our assumption that $a_2>a_1$ implies that $\mathsf{height}(R_2)>\mathsf{height}(R_1)$. This is a contradiction to our assumption that $b_1>b_2$, which implies that $\mathsf{width}(R_1)>\mathsf{width}(R_2)$. □

The above lemma, Theorem 3.6, and Dilworth's theorem together imply the following.

Corollary 3.10. All primitive 2D strings that span a primitive 2D string R can be decomposed to two sets H and V, such that

- the elements of H span R horizontally;

- the elements of V span R vertically;
- the restriction of g to $H \cup V$ is an injective function;
- each of the sets $g(H)$ and $g(V)$ is a chain.

Finally, as mentioned in the introduction, we need the following purely geometric lemma that follows from the result of Marcus and Tardos [69] on the number of 1s in an $m \times m$ binary matrix M that avoids a fixed permutation P as a submatrix.

Lemma 3.11. Consider a positive integer m and a set $\mathcal{P} \subseteq [1\,..\,m]^2$. If, for each $p \in \mathcal{P}$, the set of points of $\mathcal{P}$ that are strictly dominated by p can be partitioned into at most two chains, then $|\mathcal{P}| = \mathcal{O}(m)$.

Proof. We think of $\mathcal{P}$ as an $m \times m$ matrix $M[1\,..\,m][1\,..\,m]$, where $M[a][b] = 1$ when $(a, b) \in \mathcal{P}$ and $M[a, b] = 0$ otherwise. Next, we say that M contains a matrix P as a submatrix when P can be obtained from M by removing rows, removing columns, and changing 1s into 0s. We claim that, by the assumptions in the lemma, M does not contain the following matrix P as a submatrix:

		1	
	1		
1			
			1

To establish this, assume otherwise. Then, there exists $(a, b) \in \mathcal{P}$ and $(a_1, b_1), (a_2, b_2), (a_3, b_3) \in \mathcal{P}$ such that (a, b) strictly dominates $(a_1, b_1), (a_2, b_2), (a_3, b_3)$ and further $(a_1, b_1), (a_2, b_2), (a_3, b_3)$ create an antichain. By Dilworth's theorem, this implies that the points in $\mathcal{P}$ dominated by (a, b) cannot be partitioned into two chains, a contradiction. Thus, M indeed does not contain P as a submatrix. Because P is a permutation matrix, this implies $|\mathcal{P}| = \mathcal{O}(m)$. □

We now complete the proof of our main result with the aid of Theorem 3.11.

Theorem 3.12. An $n \times n$ 2D string has $\mathcal{O}(n^2 \log n)$ distinct quartics.

Proof. The number of distinct quartics is at most $|\mathsf{BR}|$. We then have

$$
\begin{aligned}
|\mathsf{BR}| &= \mathcal{O}(n^2 \log n) + \sum_R |\mathsf{BR}_R| && \text{(Theorems 3.2 and 3.3)} \\
&= \mathcal{O}(n^2 \log n) + \sum_R |\mathsf{occ}_{5\times 5}(R)| && \text{(Theorem 3.4)} \\
&= \mathcal{O}(n^2 \log n) + \sum_{i=1}^{n} \sum_{j=1}^{n} |\mathsf{assign}(i, j)|. && (1)
\end{aligned}
$$

To conclude the proof, it remains to show that $|\mathsf{assign}(i, j)| = \mathcal{O}(\log n)$ for all $(i, j) \in [1\,..\,n]^2$. Let $m = \lfloor \log n \rfloor$, and recall that $\mathcal{P} \subseteq [1\,..\,m]^2$ was defined as the image of f, which in turn was the restriction of g to the domain $\mathsf{assign}(i, j)$. By Theorem 3.6, we only need to show that $|\mathcal{P}| = \mathcal{O}(m)$. By Theorem 3.8 and Theorem 3.10, for each $p \in \mathcal{P}$, the set of all points of $\mathcal{P}$ that are strictly dominated by p can be partitioned into at most two chains. Thus, by Theorem 3.11 we conclude that indeed $|\mathcal{P}| = \mathcal{O}(m)$, concluding the proof. □

4 The Optimal Algorithm

In this section, we consider an $n \times n$ 2D string A over an ordered alphabet Σ and describe our $\mathcal{O}(n^2 \log n)$-time algorithm for computing distinct quartics in A. After an $\mathcal{O}(n^2 \log n)$-time preprocessing, that consists of sorting the characters and renaming them, we can assume that the entries of A are in $[1\,..\,n^2]$.

4.1 Technical Overview

Our algorithm computes thin and thick quartics separately. Here, we provide an overview of these computations.

Computation of Thin Quartics

For thin quartics, our algorithm is quite similar to the combinatorial analysis. For each position (i, j), we compute an $\mathcal{O}(\log n)$-size superset $\mathcal{C}$ of the canonical sets that have a non-empty intersection with $\mathsf{BR}(i, j)$. We do this by relating extreme occurrences of quartics with occurrences of squares in metastrings obtained by viewing the columns of $A[i\,..\,i+2^a)[1\,..\,n]$, where $a \in [1\,..\,\lfloor \log n \rfloor]$, as metacharacters. These squares can be efficiently computed and give us a handle on the sought thin quartics. Then, we compute the intersection of each canonical set in $\mathcal{C}$ with $\mathsf{BR}(i, j)$ in constant time using known tools that allow us to efficiently operate on the metastrings. We do this by fixing $(a, b) \in [1\,..\,\lfloor \log n \rfloor]^2$ and computing all quartics with height in $[2^a\,..\,2^{a+1})$ and width $[2^b\,..\,2^{b+1})$ that occur at position (i, j) of A, of the form $P^{2y,2}$, $P^{2y,4}$, $P^{2,x}$, and $P^{4,4x}$, for a primitive 2D string P and $x, y \geq 1$. A detailed description can be found in Section 4.3.

Computation of Thick Quartics

For thick quartics, our algorithmic approach follows our combinatorial approach in a more relaxed sense. The main technical challenge is to compute, for each position (i, j), an $\mathcal{O}(\log n)$-size set $\mathcal{R}$ of primitive 2D strings R, such that $\mathsf{assign}(i, j) \subseteq \mathcal{R}$. Then, those supersets can be postprocessed as in [25] in time linear in their total size to yield the sought distinct thick quartics. The computation of $\mathcal{R}$ is split into two major steps which we outline next. A detailed description can be found in Section 4.4.

Skyline Computation. First, we compute a set $\mathcal{S}$ of skyline primitive 2D strings such that $S \in \mathcal{S}$ when (a) $S^{5,5}$ has an occurrence anchored at position (i, j), and (b) there is no other primitive 2D string T with $g(T) \geq g(S)$ such that $T^{5,5}$ has an occurrence anchored at position (i, j). This part of the proof is quite technical: it heavily relies on the analysis of periodicity for 1D (meta)strings and, roughly speaking, on the analysis of the evolution of the horizontal periodic structure of a 2D string as rows are appended to it. We show that, for each $a \in [1\,..\,\lfloor \log n \rfloor]$, there is a single candidate $h \in [2^a\,..\,2^{a+1})$ to be considered as the height of an element of $\mathcal{S}$. Then, using runs in 1D metastrings whose origins in A have sufficient overlap and bit-tricks, we can compute the widest 2D string S with height h such that an occurrence of $S^{5,5}$ is anchored at position (i, j) in constant time (using batched computations), if one exists.

Computation of Dominated 2D strings. This turned out to be the most challenging part of our approach. For this exposition, let us treat $\Sigma^{*,*}$ as a partially ordered set, in the order of decreasing widths. Let $\mathcal{S} = \{S_1, \ldots, S_\ell\}$ in accordance with this order. To obtain $\mathcal{R}$ from $\mathcal{S}$, we need to add

to it the 2D strings $R \in \mathsf{assign}(i,j) \setminus \mathcal{S}$. By the construction of $\mathcal{S}$ we know that there exists $S \in \mathcal{S}$ such that $g(R) \leq g(S)$.

Our combinatorial analysis implies that each $R \in \mathsf{assign}(i,j)$ spans each element $S \in \mathcal{S}$ for which $g(S)$ stricly dominates $g(R)$ either vertically or horizontally. It turns out that if R spans all of these elements of $\mathcal{S}$ either vertically or horizontally, it is easy to compute it efficiently. This is, unfortunately, not the case in general. However, we observe that R spans vertically (resp. horizontally) a contiguous subset of $\mathcal{S}$. We show that the problem boils down to computing the union, over all k, of sets I_k, where I_k contains exactly those primitive 2D strings that span S_{k-1} vertically and span S_k horizontally. Crucially, we observe that due to a strong form of transitivity of the spanning property, the intersection of any two such I_k and $I_{k'}$ consists of a number of the smallest elements of both (i.e., their longest common prefix if viewed as strings). Hence, by computing the elements of I_k from the largest to the smallest, we can stop whenever we encounter an element that has already been reported by this procedure. This allows us to reduce the computation of $\mathcal{R}$ to the problem of efficiently computing sets I_k. To this end, we prove that an $\mathcal{O}(\log n)$-bits representation of the evolution of the periodic structure of certain fragments of A as rows and columns are appended to them suffices for inferring I_k; we then use tabulation to infer it efficiently.

4.2 Preprocessing

For each $a \in [1\,..\,\lfloor \log n \rfloor]$, for each $i \in [1\,..\,n-2^a]$, let us denote by H_i^a the metastring obtained from $A[i\,..\,i+2^a)[1\,..\,n]$ by viewing each column as a metacharacter; let us denote the collection of those strings by $\mathcal{H}$. Similarly, for each $b \in [1\,..\,\lfloor \log n \rfloor]$, for each $j \in [1\,..\,n-2^b]$, let us denote by V_j^b the metastring obtained from $A[1\,..\,n][j\,..\,j+2^b)$ by viewing each row as a metacharacter; let us denote the collection of those strings by $\mathcal{V}$. See Figure 6. Overall, we have $\mathcal{O}(n \log n)$ metastrings, each of length n. The metastrings in $\mathcal{H}$ and $\mathcal{V}$ can be computed in $\mathcal{O}(n^2 \log n)$ time in total as follows. It suffices to explain how to compute all strings of the form H_i^a as the computation of the strings of the form V_j^b is symmetric. For any i, H_i^1 can be computed by radix-sorting two-tuples of characters and renaming them, while H_i^a for $a > 1$ can be similarly computed using two-tuples consisting of characters from H_i^{a-1} and $H_{i+2^{a-1}}^{a-1}$.

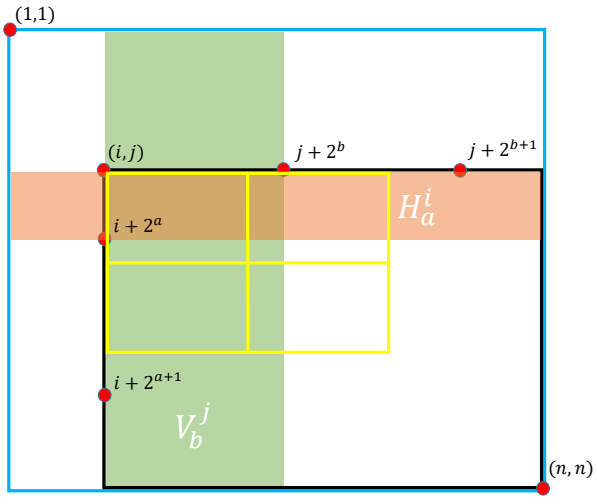


Figure 6: A is presented in blue. The black rectangle corresponds to $A[i\,..\,n][j\,..\,n]$. The orange rectangle corresponds to H_i^a. The green rectangle corresponds to V_j^b. A quartic in $\mathsf{BR}(i,j) \cap C_{(a,b)}$ is shown in yellow.

All runs in a string of length n over an alphabet of integers from $[n^c]$, for some constant c (for short: polynomial alphabet), can be computed in $\mathcal{O}(n)$ time [42,63]. We compute the runs in each of the $\mathcal{O}(n \log n)$ length-n strings in $\mathcal{H}$ and $\mathcal{V}$ in $\mathcal{O}(n^2 \log n)$ time in total.

Let us now define the following primitive operations for fragments of a text T.

- An internal pattern matching query takes as input two fragments U and V of a text, and returns all occurrences of U in V in the form of $|V|/|U|$ arithmetic progressions with difference $\mathsf{per}(U)$; such a query is denoted by $\mathsf{IPM}(U, V)$.
- A 2-period query takes as input a substring U of a text, checks if U is periodic and, if so, it also returns U's period.
- A periodic extension query takes as input a periodic fragment of a text and returns the (unique) run that has the same period and contains it [61].

We preprocess each of the strings in $\mathcal{H}$ and $\mathcal{V}$ according to the following theorem, in $\mathcal{O}(n^2 \log n)$ time in total.

Theorem 4.1 ([61,62]). A string T of length n over a polynomial alphabet can be preprocessed in $\mathcal{O}(n)$ time so that queries of the form $\mathit{IPM}(U, V)$, for any two fragments U and V of T, can be answered in $\mathcal{O}(|V|/|U|)$ time, and 2-period queries and period extension queries can be answered in $\mathcal{O}(1)$ time.

We also apply the following lemma to each of the strings in $\mathcal{H}$ and $\mathcal{V}$ in $\mathcal{O}(n^2 \log n)$ time in total.

Lemma 4.2. Given a string T of length n over a polynomial alphabet, we can compute the length of the longest square that occurs at each position of T in $\mathcal{O}(n)$ time. In addition, a query of the form (i, x), asking for the primitive squares that occur at some position i and have length in $[2^x \,..\, 2^{x+1})$ can be answered in $\mathcal{O}(1)$ time after an $\mathcal{O}(n)$-time preprocessing.

Proof. We first compute all distinct squares in T in $\mathcal{O}(n)$ time. [19,35]. Then, we build the suffix tree of T in $\mathcal{O}(n)$ time [44], with nodes weighted by string-depth. We say that a node v is the weighted ancestor of a node u at depth ℓ if v is the highest ancestor of u with weight of at least ℓ. Belazzougui et al. [20] have shown that the suffix tree of a length-n string can be preprocessed in $\mathcal{O}(n)$ time so that weighted ancestor queries (with weights equal to string-depths) can be answered in $\mathcal{O}(1)$ time. After this preprocessing, we compute the locus of each square (in decreasing order with respect to length) in the suffix tree of T in $\mathcal{O}(1)$ time in total and make the corresponding node explicit. This takes $\mathcal{O}(n)$ time in total. Additionally, we mark the nodes corresponding to primitive squares with blue and the nodes corresponding to non-primitive squares with red—we can distinguish between the two cases in $\mathcal{O}(1)$ time using a 2-period query (see Theorem 4.1).

Then, the length of the longest square that occurs at a position i is equal to the string-depth of the nearest ancestor of the node with path-label $T[i \,..\, n]$ that is marked (either red or blue). Similarly, the lengths of the primitive squares that occur at some position i and have length in $[2^x \,..\, 2^{x+1})$ can be computed in constant time given the string-depths of the at most two nearest ancestors of the node v with path-label $T[i \,..\, i + 2^{x+1} - 1)$ that are marked with blue; node v can be computed in $\mathcal{O}(1)$ time with a weighted ancestor query. The nearest blue or red marked node for each node in a tree can be precomputed in $\mathcal{O}(n)$ time. This concludes the proof. □

4.3 Computation of Thin Quartics

4.3.1 Computation of $\mathcal{O}(\log n)$ Canonical Sets

The following observation and fact are counterparts of Theorem 3.1 that are easier to exploit computationally.

Observation 4.3. An occurrence of a quartic Q with height in $[2^a\,..\,2^{a+1})$, where $a \in \mathbb{Z}_+$, at a position (i, j) of A implies an occurrence of a square of length $\mathit{width}(Q)$ at position j in string H_i^a.

Fact 4.4. An occurrence of a square with length in $[2^b\,..\,2^{b+1})$, where $b \in \mathbb{Z}_+$, at some position j in string H_i^a, where $a, i \in \mathbb{Z}_+$, implies that no quartic with height in $[2\,..\,2^{a-1})$ and width in $[2\,..\,2^{b-1})$ can have an extreme occurrence at position (i, j) of A.

Proof. Let us denote the specified square by UU. The claim follows by the observation that any quartic with height in $[2\,..\,2^{a-1})$ and width in $[2\,..\,2^{b-1})$ that occurs at position (i, j) of A must also have an occurrence at position $(i, j + |U|)$. □

We next combine Theorems 4.3 and 4.4 to efficiently compute a small superset of the canonical sets that have a non-empty intersection with $\mathsf{BR}(i, j)$ for a fixed position (i, j).

Lemma 4.5. After an $\mathcal{O}(n^2 \log n)$-time preprocessing of A, we can compute, for any position (i, j) of A, an $\mathcal{O}(\log n)$-size superset of the canonical sets that have a non-empty intersection with $\mathit{BR}(i, j)$ in $\mathcal{O}(\log n)$ time.

Proof. Let us fix a position (i, j). For each $a \in [1\,..\,\lfloor \log n \rfloor]$, let ℓ_a be the length of the longest square at position j in H_i^a; all of these lengths can be retrieved in $\mathcal{O}(\log n)$ time after an $\mathcal{O}(n^2 \log n)$-time preprocessing due to Theorem 4.2. The sequence $\ell_1, \ldots, \ell_{\lfloor \log n \rfloor}$ is non-increasing since, for each square in H_i^x, where $x \in (1\,..\,\lfloor \log n \rfloor]$, there is a square of the same length at the same position in H_i^y for each $y \in [1\,..\,x)$. Now, let α and β be integers such that the canonical set $C_{(\alpha,\beta)}$ has a non-empty intersection with $\mathsf{BR}(i, j)$. By Theorem 4.3, (α, β) must by dominated by some point in $Z = \{(a, \lfloor \log \ell_a \rfloor) : a \in [1\,..\,\lfloor \log n \rfloor]\}$. Let Z' be the union of Z with the set of all points that are weakly dominated by some point in Z, but are not strongly dominated by any point in Z; the elements of Z' form a staircase of size $\mathcal{O}(\log n)$. Then, by Theorem 4.4, (α, β) must be in the set $\{(a - x, b - y) : (a, b) \in Z' \text{ and } x, y \in \{0, 1, 2\}\}$, which is of size $\mathcal{O}(\log n)$ and can be naively computed in $\mathcal{O}(\log n)$ time given $\ell_1, \ldots, \ell_{\lfloor \log n \rfloor}$. See Figure 7. □

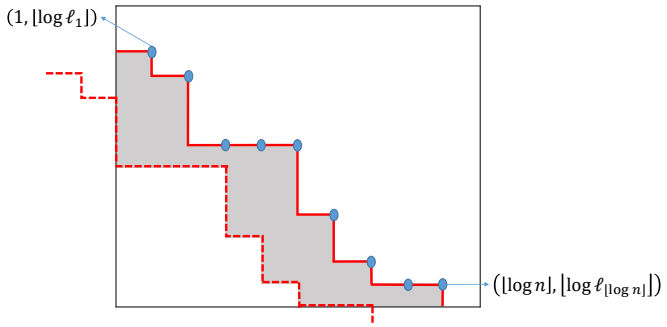


Figure 7: The elements of Z are the blue points. The elements of Z' are the points on the solid red line. The dashed red line corresponds to the set of points $\{(a - 2, b - 2) : (a, b) \in Z'\}$. The set $\{(a - x, b - y) : (a, b) \in Z' \text{ and } x, y \in \{0, 1, 2\}\}$ is the grey area.

4.3.2 Thin Quartics from a Canonical Set

Fix $(a, b) \in [1 \,..\, \lfloor \log n \rfloor]^2$ such that (a, b) corresponds to a canonical set that was computed earlier by Theorem 4.5. We wish to find all thin quartics with height in $[2^a \,..\, 2^{a+1})$ and width $[2^b \,..\, 2^{b+1})$ that occur at position (i, j) of A, recall that we have at most 10 such quartics by Lemma 3.2. That is, those quartics Q, such that $g(Q) = (a, b)$ and Q equals one of $P^{2y,2}$, $P^{2y,4}$, $P^{2,x}$, and $P^{4,4x}$, for a primitive 2D string P and $x, y \geq 1$ (so $P^{2,2}$ will be counted twice). Below, we show how to compute all quartics that fall in the first two cases, i.e., they are of the form $P^{2y,2}$, $P^{2y,4}$; the remaining ones can be computed symmetrically.

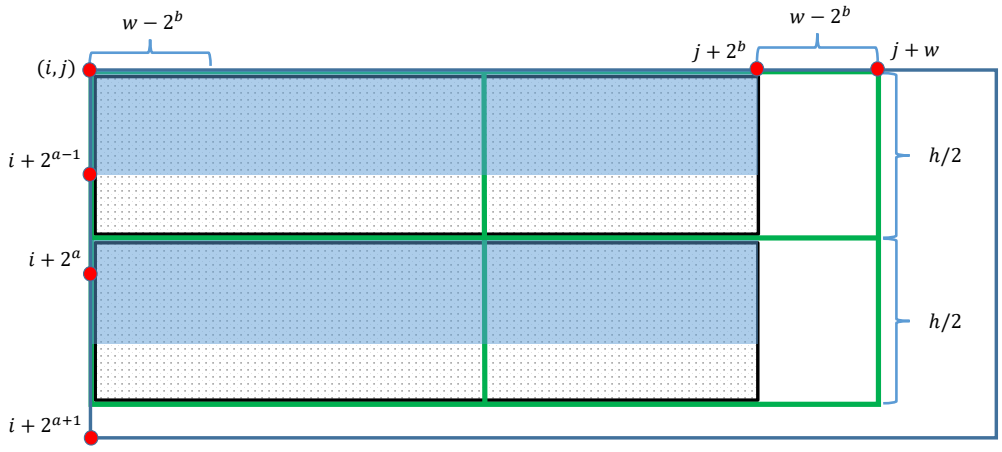
Computation of Three Candidate Widths. Let us set $S := H^a_i$.

- Similarly to Theorem 4.3, an occurrence of a quartic $Q \in C_{(a,b)}$ of the form $P^{2y,2}$ at position (i, j), where P is a primitive 2D string, implies an occurrence of a primitive square of length $\mathsf{width}(Q)$ at position j of S. At most two primitive squares with length in $[2^b \,..\, 2^{b+1})$ start at any position j of S, cf. Theorem 2.3.
- Analogously, an occurrence of a quartic $Q \in C_{(a,b)}$ of the form $P^{2y,4}$ at position (i, j), where P is a primitive 2D string, implies an occurrence of a string U^4, where U is a primitive string, at position j of S. There is at most one primitive string U such that U^4 occurs at position j of S and $4 \cdot |U| \in [2^b \,..\, 2^{b+1})$, due to the periodicity lemma (Theorem 2.1), since $|U| < 2^{b-1}$ is a period of $S[j \,..\, j + 2^b)$.

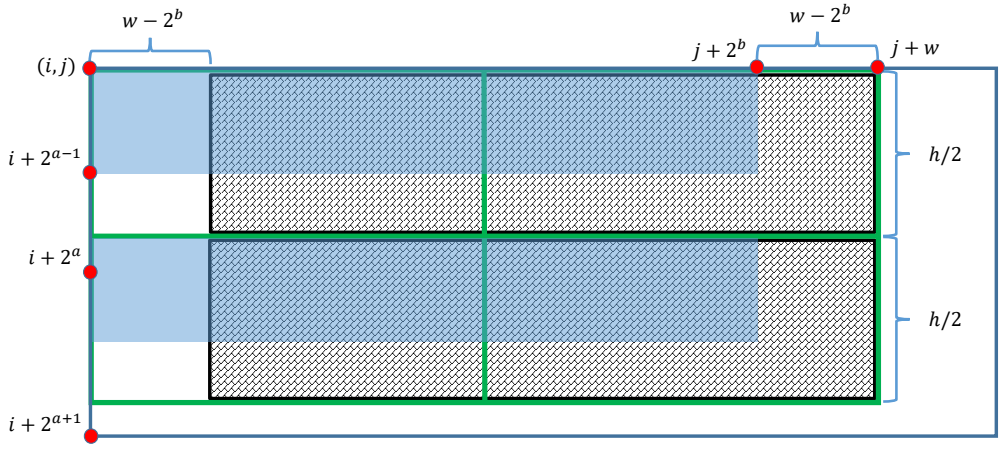
The above analysis implies that there are three candidate widths for quartics in $\mathsf{BR}(i, j) \cap C_{(a,b)}$. We next show how to compute them efficiently. The primitive squares that occur at position j of S and have length in $[2^b \,..\, 2^{b+1})$ are computed in $\mathcal{O}(1)$ time due to Theorem 4.2. Additionally, we compute a primitive string U such that U^4 occurs at position j of S and $4 \cdot |U| \in [2^b \,..\, 2^{b+1})$, if one exists, as follows. We first perform, in $\mathcal{O}(1)$ time, a 2-period query for $S[j \,..\, j + 2^b)$. If this fragment is not periodic there is no such U. Otherwise, we have a candidate length for U. We check if U^4 occurs at position j of S by performing a periodic extension query for $S[j \,..\, j + 2^b)$. Thus, in $\mathcal{O}(n^2 \log n)$ time in total, we obtain three candidate widths for each of the $\mathcal{O}(n^2 \log n)$ considered intersections of some $\mathsf{BR}(i, j)$ and some canonical set $C_{(a,b)}$.

Processing each Candidate Width. We next show how to process one of the three candidate widths, say w, in time $\mathcal{O}(1)$. Observe that $G := A[i \,..\, i + 2^{a-1})[j \,..\, j + 2^b)$ has to be contained in the top half of the sought quartic(s). We start by computing G's occurrences in $A[i \,..\, i + 2^{a+1})[j \,..\, j + 2^b)$. This can be done in $\mathcal{O}(1)$ time by employing an IPM query for fragments of V^b_j. It would then be enough to consider, for each occurrence $A[i + h/2 \,..\, i + h/2 + 2^{a-1})[j \,..\, j + 2^b)$ of G the integer h as a candidate height for the sought quartic(s). Given two integers w and h, we can check whether there is a quartic with height h and width w at position (i, j) in $\mathcal{O}(1)$ time by checking whether the following equalities hold:

- $A[i \,..\, i + h/2)[j \,..\, j + 2^b) = A[i + h/2 \,..\, i + h)[j \,..\, j + 2^b)$;
- $A[i \,..\, i + h/2)[j + w - 2^b \,..\, j + w) = A[i + h/2 \,..\, i + h)[j + w - 2^b \,..\, j + w)$;
- $A[i \,..\, i + 2^a)[j \,..\, j + w/2) = A[i \,..\, i + 2^a)[j + w/2 \,..\, j + w)$;
- $A[i + h - 2^a \,..\, i + h)[j \,..\, j + w/2) = A[i + h - 2^a \,..\, i + h)[j + w/2 \,..\, j + w)$.



(a) The black textured rectangles correspond to $A[i\,..\,i+h/2)[j\,..\,j+2^b)$ and $A[i+h/2\,..\,i+h)[j\,..\,j+2^b)$.



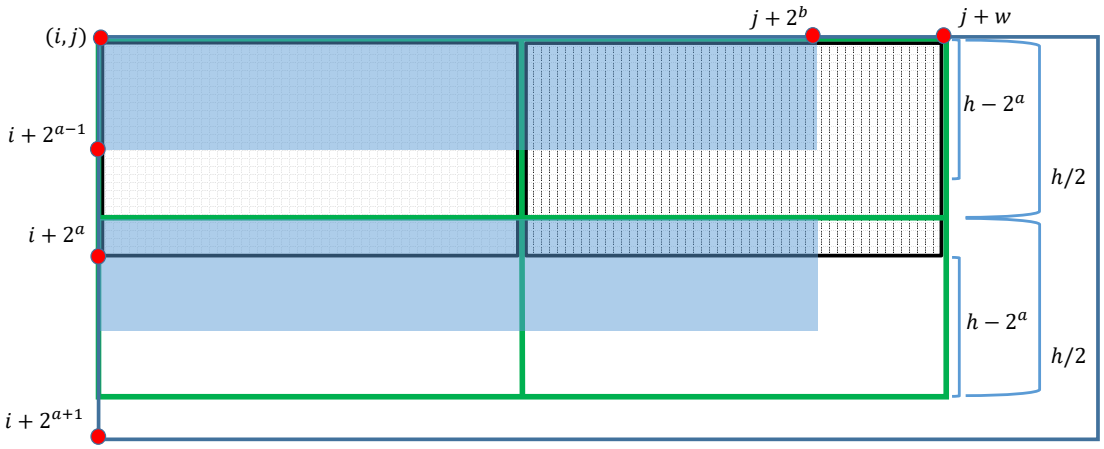
(b) The black textured rectangles correspond to $A[i\,..\,i+h/2)[j+w-2^b\,..\,j+w)$ and $A[i+h/2\,..\,i+h)[j+w-2^b\,..\,j+w)$.

Figure 8: In both (a) and (b) $G := A[i\,..\,i+2^{a-1})[j\,..\,j+2^b)$ is the top rectangle filled with blue that equals to $A[i+h/2\,..\,i+h/2+2^{a-1})[j\,..\,j+2^b)$, i.e., the bottom rectangle filled with blue. The sought quartic of height h and width w is shown in green.

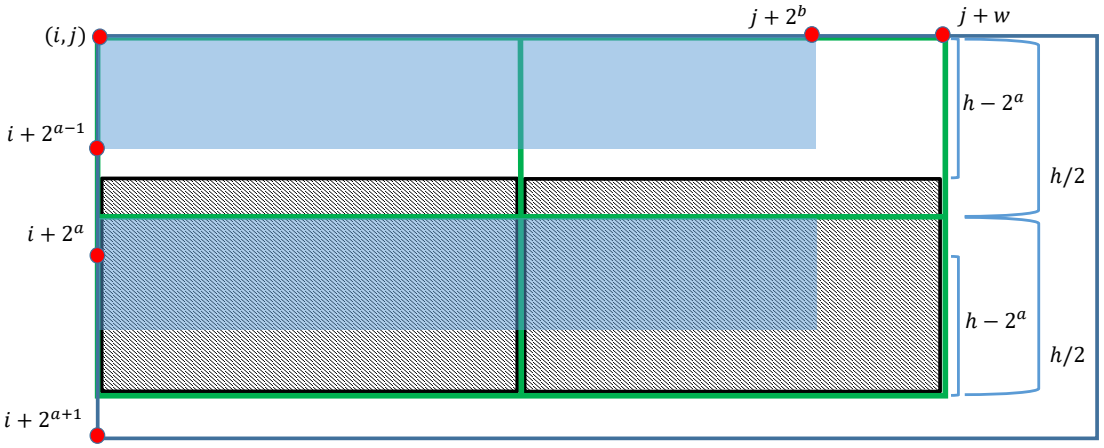
The latter can be done in $\mathcal{O}(1)$ time using 4 IPM queries, one for each of the fragments: V^b_j, $V^b_{j+w-2^b}$, H^a_i, and $H^a_{i+h-2^a}$. Note that, if the first two equalities hold then the two left quarters of the sought quartic are equal and the two right quarters of the sought quartic are equal, see Figure 8. Moreover, if the last two equalities hold then the two upper quarters of the sought quartic are equal and the two lower quarters of the sought quartic are equal, see Figure 9.

However, if G has a small vertical period, it may have more than a constant number of occurrences in $A[i\,..\,i+2^{a+1})[j\,..\,j+2^b)$. One can still process all of them in constant time, as we describe in detail below. In what follows, we do not explicitly mention that before reporting any quartic obtained from either of the cases, we first verify that it is in $C_{(a,b)}$, i.e., its height is in $[2^a\,..\,2^{a+1})$ and its width is in $[2^b\,..\,2^{b+1})$.

1. If the IPM query returns at most 10 occurrences, we verify the candidate height h in $\mathcal{O}(1)$ time as explained above.



(a) The black textured rectangles correspond to $A[i\,..\,i+2^a)[j\,..\,j+w/2)$ and $A[i\,..\,i+2^a)[j+w/2\,..\,j+w)$.



(b) The black textured rectangles correspond to $A[i+h-2^a\,..\,i+h)[j\,..\,j+w/2)$ and $A[i+h-2^a\,..\,i+h)[j+w/2\,..\,j+w)$.

Figure 9: In both (a) and (b) $G := A[i\,..\,i+2^{a-1})[j\,..\,j+2^b)$ is the top rectangle filled with blue that equals to $A[i+h/2\,..\,i+h/2+2^{a-1})[j\,..\,j+2^b)$, i.e., the bottome rectangle filled with blue. The sought quartic of height h and width w is shown in green.

2. Suppose that the IPM query returns more than 10 occurrences. These occurrences are represented as a constant number of arithmetic progressions such that the difference equals the vertical period of G, which we denote by p. Note that, if we consider G as a metastring by viewing its rows as metacharacters then each arithmetic progression corresponds to a run with period p of this metastring.

   (a) Let us first focus on the case where p is not a vertical period of the sought quartic(s). We obtain $\mathcal{O}(1)$ candidate heights from the returned arithmetic progressions as follows, distinguishing between two cases:

      i. If the top-left quarter of the sought quartic(s) has vertical period p, we argue next that it suffices to consider the differences between i and the first element of each other (maximal) arithmetic progression with difference p, denoted by $h/2$, i.e., the height of the quarter(s), and as a result, we consider h as a candidate height of the

quartic(s). See Figure 10. Suppose that there is such a quartic $A[i\,..\,i+h)[j\,..\,j+w)$. Observe that the occurrences of G at positions (i,j) and $(i+h/2,j)$ must belong to different arithmetic progressions, as otherwise, the whole sought quartic(s) would have vertical period p. Then, the runs corresponding to these arithmetic progressions must overlap. However, two runs with period p cannot overlap by p or more positions and it hence suffices to consider only the first element of the arithmetic progression that corresponds to the rightmost of the two considered runs.

ii. In the complementary case it suffices to consider as candidate heights the differences between the last element of the arithmetic progression that contains position i in V_j^b and the last element of every other returned arithmetic progression. This is sufficient because by the assumption on p not being a vertical period of the top-left quarter the last element of the former arithmetic progression corresponds to an occurrence of G that is fully within the top-left quarter.

In the end, we verify each candidate height as in the case where the IPM query returns at most 10 occurrences.

(b) Finally, we need to consider the case where the sought quartic(s) has/have a vertical period equal to p. We already know the candidate horizontal period, which is either $w/2$ or $w/4$, depending on whether the origin of the candidate width w is a primitive square or a string U^4 for a primitive U. This means that we already know the primitive 2D string P for which we look for quartic(s) of the form $P^{2y,x}$, where $x \in \{2,4\}$. Next, we employ periodic extension queries for V_j^b and $V_{j+w-2^b}^b$ to compute the maximum t such that $A[i\,..\,t)[j\,..\,j+w)$ has a vertical period equal to p. Let $y_{\max} := \lfloor t/2p \rfloor$. Then, the quartics of the form $P^{2y,x}$ that occur at position (i,j) are those for $y \in [1\,..\,y_{\max}]$. Observe that only the quartic of maximal height among these quartics may have an extreme occurrence at position (i,j), as all the other ones have another occurrence at position $(i+p,j)$. See Figure 11.

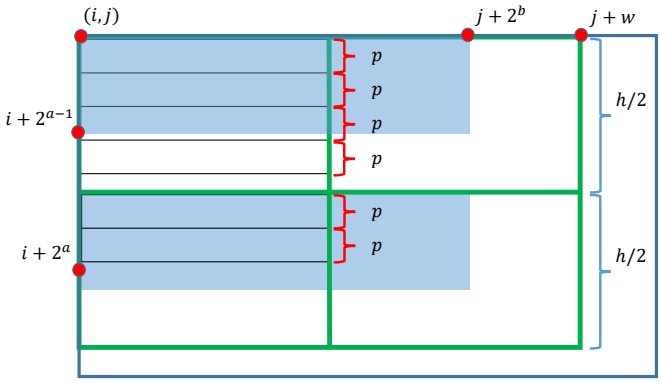


Figure 10: $G := A[i\,..\,i+2^{a-1})[j\,..\,j+2^b)$ is the top rectangle filled with blue and a candidate quartic $A[i\,..\,i+h)[j\,..\,j+w)$ is shown in green, where the top-left quarter has vertical period p. The height $h/2$ corresponds to the difference between i and some occurrence of G.

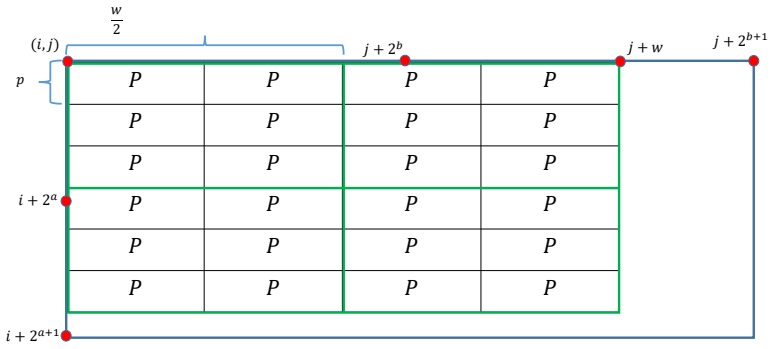


Figure 11: $A[i\,..\,i+2^{a+1})[j\,..\,j+2^{b+1})$ is the blue border rectangle and a candidate quartic of maximal height, i.e., $P^{6,4}$, is shown in green.

4.4 Computation of Thick Quartics

4.4.1 Computation of Supersets of **assign**(i,j)

In this subsection, we show how to compute, for each position (i,j) of A, an $\mathcal{O}(\log n)$-size superset of $\mathsf{assign}(i,j)$, in $\mathcal{O}(n^2\log n)$ time in total.

Let us fix a position (i,j) of A. We say that a primitive 2D string R is a skyline 2D string (for position (i,j)) if and only if

- $R^{5,5}$ has an occurrence anchored at position (i,j), and
- There is no other primitive 2D string P, such that $g(P)$ dominates $g(R)$ and $P^{5,5}$ has an occurrence anchored at position (i,j).

Note that a skyline 2D string R does not have to be in $\mathsf{assign}(i,j)$.

Computation of Skyline 2D Strings. We wish to efficiently compute, for each $a\in[1\,..\,\lfloor\log n\rfloor]$, the widest 2D primitive string R, if there is any, with $\mathsf{height}(R)\in[2^a\,..\,2^{a+1})$ and for which there is an occurrence of $R^{5,5}$ anchored at position (i,j). To this end, we start with a combinatorial result about highly periodic runs in strings V_j^b for a fixed j and variable b.

Lemma 4.6. Let (i,j) be a position of A. Suppose that there are primitive strings U and V with $|U|<|V|$ and (not necessarily distinct) integers b_1 and b_2 such that U^3 occurs at position i of $V_j^{b_1}$ and V^3 occurs at position i of $V_j^{b_2}$. Then, we have that $2|U|\le|V|$.

Proof. Towards a contradiction, suppose that $|V|<2|U|$. Let $b:=\min\{b_1,b_2\}$ and $S:=V_j^b$.

Case I: $b_1\le b_2$. By our assumptions, $|V|+|U|<3|U|$. Then, as $|V|$ is not a multiple of $|U|$, $S[i+|V|\,..\,i+|V|+|U|)$ must equal some non-trivial rotation of U (defined in Section 2). On the other hand, $S[i+|V|\,..\,i+|V|+|U|)=S[i\,..\,i+|U|)=U$ as $|V|$ is a period of $S[i\,..\,i+3|V|)$. We obtain a contradiction, as the primitive string U cannot match any of its non-trivial rotations.

Case II: $b_1>b_2$. The period p of $S[i\,..\,i+3|U|)$ divides $|U|$. Our assumptions imply that $|V|+p<3|U|<3|V|$. Further, $S[i+|V|\,..\,i+|V|+p)=S[i\,..\,i+p)$ as $|V|$ is a period of $S[i\,..\,i+3|V|)$. Since $S[i\,..\,i+p)$ is primitive, it cannot match any of its non-trivial rotations, and hence $S[i\,..\,i+|V|)=S[i\,..\,i+p)^k$ for some k. This contradicts the primitivity of V and thus concludes the proof. □

The above lemma implies that there is at most one candidate for the height of R. We next show how to efficiently compute this candidate or conclude that none exists. We compute as a batch of $\mathcal{O}(n \log n)$ candidates for a fixed j and all the $\mathcal{O}(n \log n)$ choices of i and a.

Lemma 4.7. Let $j \in [1 \,..\, n]$. In $\mathcal{O}(n \log n)$ time in total, we can compute, for each $i \in [1 \,..\, n]$, for each $a \in [1 \,..\, \lfloor \log n \rfloor]$, the unique integer h with $\lfloor \log h \rfloor = a$, for which there exists some $b \in [1 \,..\, \lfloor \log n \rfloor]$ such that a string U^5, where U is a primitive string of length h, occurs at position $i - 2h$ of V_j^b. The subinterval I_h of $[1 \,..\, \lfloor \log n \rfloor]$ that contains exactly those integers b for which this holds is also returned alongside h.

Proof. For each $b > 1$, for each run $D = V_j^b[x \,..\, y]$ with period p, there is a unique run $\pi(D)$ in string V_j^{b-1} that contains $V_j^{b-1}[x \,..\, y]$ and whose period divides p. It can be computed in $\mathcal{O}(1)$ time using a periodic extension query in V_j^{b-1}.

We build a forest $\mathcal{F}$ whose nodes are runs D with $|D| \geq 5 \cdot \mathsf{per}(D)$ of V_j^b, for $b \in [1 \,..\, \lfloor \log n \rfloor]$. Each run of V_j^1 is a root, while the parent of any other run D is $\pi(D)$. We preprocess $\mathcal{F}$ in $\mathcal{O}(n \log n)$ time so that each run/node in $\mathcal{F}$ stores its depth as well as a special pointer to its lowest ancestor that has a different period.

We process the runs that are represented in this forest in a left-to-right manner. That is, we consider positions $i = 1, 2, \ldots, n$ in this order while maintaining the information about runs in all strings V_j^b, for $b \in [1 \,..\, \lfloor \log n \rfloor]$, simultaneously. A run D is activated $2 \cdot \mathsf{per}(D)$ positions after it starts and deactivated $3 \cdot \mathsf{per}(D) - 1$ positions before it ends. When processing a position $i \in [1 \,..\, n]$, we first deactivate runs as necessary (bottom-up) and then activate runs as necessary (top-down), maintaining the leaves of the subforest induced by the active nodes. This guarantees that all ancestors of an active node are also active. Then, we traverse the subforest induced by the active nodes by following special pointers, starting from the leaves. The number of traversed special pointers is $\mathcal{O}(\log n)$ due to Theorem 4.6: for any $k \in [1 \,..\, \lfloor \log n \rfloor]$, all active runs with periods from $[2^k \,..\, 2^{k+1})$ have the same period p, thus forming a single path in the subforest, and the traversal can be implemented in time proportional to the number of such paths. Each traversed special pointer gives us a period p and a subinterval $I_h \subseteq [1 \,..\, \lfloor \log n \rfloor]$ containing all b such that there is an active run D at position i in V_j^b with $\mathsf{per}(D) = p$. The value of p is stored at the tail of the special pointer, and I_h consists of the depths of all nodes between its tail (inclusive) and head (exclusive). □

Processing the candidate vertical period. Let us now fix a position (i, j) and an $a \in [1 \,..\, \lfloor \log n \rfloor]$ for which Theorem 4.7 returns a candidate h, accompanied by a subinterval I_h of $[1 \,..\, \lfloor \log n \rfloor]$. Now, we need to compute the widest primitive 2D string R with $\mathsf{height}(R) = h$ such that $R^{5,5}$ has an occurrence anchored at position (i, j) if one exists.

Let $\ell = \lfloor \log(5h) \rfloor$. Recall that $\mathcal{H}$ is the collection of all strings H_i^a. There is a subset $\mathcal{C}$ of $\mathcal{H}$ of size at most three, such that the union of the fragments of A from which the metastrings in $\mathcal{C}$ originate equals $A[i - 2h \,..\, i + 3h)[1 \,..\, n]$, and each of these fragments overlaps with the next one (in the order induced by the indices of their topmost rows) by at least h rows. We can assume that $\mathcal{C}$ contains $T := H_{i-2h}^\ell$ and the elements of some subset of $\{H_{i+2^\ell-h}^\ell, H_{i+3h-1-2^\ell}^\ell\}$.

In a preprocessing step, we apply the following lemma with $k = 2$ to each of the strings in $\mathcal{H}$ in $\mathcal{O}(n^2 \log n)$ time in total.

Lemma 4.8. Given a string S of length n and a non-negative integer k, we can compute, in $\mathcal{O}(n)$ time, for each $j \in [1 \,..\, n]$, a bitvector $\beta(S, j)$ of size $\lfloor \log n \rfloor$, such that the b-th bit of $\beta(S, j)$ is set if and only if there exists a primitive string U such that $S[j - k|U| \,..\, j + 3|U|) = U^{k+3}$ and $\lfloor \log |U| \rfloor = b$.

Proof. We perform a line-sweeping algorithm on S that processes all runs whose length is at least $k+3$ times larger than their period. These runs can be computed in $\mathcal{O}(n)$ time [18, 63]. Now, let us start by setting $\beta(S,j)$ to be an all-zeroes bitvector of size $\lfloor \log n \rfloor$. As we sweep over S, in a left-to-right manner, for each of the specified runs $D = S[x\,..\,y]$, we set the $\lfloor \log(\mathsf{per}(D)) \rfloor$-th bit of the maintained bitvector, i.e., $\beta(S,j)$, when position $x + k \cdot \mathsf{per}(D)$ in S is processed and unset it when position $y - 3 \cdot \mathsf{per}(D) + 1$ in S is processed. For each j, we set $\beta(S,j)$ to be equal to the maintained bitvector just after position j in S is processed. The correctness of this approach follows by Theorem 4.6, which implies that, for each position j in S, for each $a \in [1\,..\,\lfloor \log n \rfloor]$ there is at most one run D at a time such that $S[j\,..\,j+3\cdot\mathsf{per}(D))$ is contained in D and $\lfloor \log(\mathsf{per}(D)) \rfloor = a$. □

Observation 4.9. If we have an occurrence of a 2D string $R^{5,5}$ for a primitive 2D string R of height h anchored at position (i,j), then, for each $X \in \mathcal{C}$, the $\lfloor \log(\mathit{width}(R)) \rfloor$-th bit of $\beta(X,j)$ is 1.

To facilitate the efficient computation of the sought width, we need a converse version of the above statement.

Lemma 4.10. Suppose that the b-th bit is set in $\beta(X,j)$ for all $X \in \mathcal{C}$. Let U be a primitive string such that $T[j-2|U|\,..\,j+3|U|) = U^5$ and $\lfloor \log |U| \rfloor = b$. Then, if the vertical period of $A[i-2h\,..\,i+3h)[j\,..\,j+3|U|)$ is h, $R := A[i-2h\,..\,i-h)[j\,..\,j+|U|)$ is a primitive 2D string and $R^{5,5}$ has an occurrence anchored at position (i,j).

Proof. Let $w := |U|$. By the assumptions of the lemma and the fact that $\mathsf{height}(T) > 2h$, the horizontal and vertical periods of $A[i-2h\,..\,i-2h+\mathsf{height}(T))[j-2w\,..\,j+3w)$ are w and h, respectively. The primitivity of R is immediate.

Now, note that if $\mathcal{C} = \{T\}$ we are already done as this can only be the case if $5h = 2^\ell$. Henceforth, we can thus suppose that this is not the case. Consider $Y \in \mathcal{C}$ such that the fragments of A from which T and Y were built (as metastrings) overlap by at least h rows, and let O be this overlap. It readily follows that the horizontal period of $O[1\,..\,h)[j-2w\,..\,j+3w)$ is w. Further, by an argument analogous to that in the proof of Theorem 4.6, w must coincide with the length of the primitive string V such that V^5 has an occurrence at position $i-2|V|$ of Y and satisfies $\lfloor \log |V| \rfloor = b$. If $\mathcal{C}$ also contains a third element X, we also apply the same reasoning to X and Y. In either case, we conclude that $A[i-2h\,..\,i+3h)[j-2w\,..\,j+3w)$ has horizontal period w. Since the vertical period of $A[i-2h\,..\,i+3h)[j\,..\,j+3w)$ is h, we have an occurrence of $R^{5,5}$ anchored at position (i,j). This completes the proof of the lemma. □

Now, our first goal is to compute a set B that contains three candidates for $\lfloor \log(\mathsf{width}(R)) \rfloor$. Initialize B as the empty set. Let $I = [b_1\,..\,b_2]$ be the interval returned by the application of Theorem 4.7 together with candidate h. Let β^* be the bitvector obtained by an AND operation performed on the bitvectors $\beta(X,j)$ for all $X \in \mathcal{C}$. By Theorem 4.9, we only need to consider indices of bits of β^* that are set. If the b-th bit of β^* is set and $b \in [b_1\,..\,b_2)$ then the requirements of Theorem 4.10 are satisfied. All such indices b can be computed by applying a bitmask whose x-th bit is set if and only if $x \in [b_1\,..\,b_2)$. Our first candidate is the index b^* of the least significant set bit of the obtained vector β^*, if there is one; we insert it to B. Further, observe that we cannot have $\lfloor \log(\mathsf{width}(R))) \rfloor \in [1\,..\,b_1-1) \cup (b_2\,..\,\lfloor \log n \rfloor]$ since the vertical period of $A[i-2h\,..\,i-h)[j\,..\,j+\mathsf{width}(R))$ is not h. Finally, insert to B each $y \in \{b_1-1, b_2\}$ if the y-th bit of β^* is set.

If B is empty there is nothing to be done as there is no primitive 2D string R to be returned. We thus assume that B is non-empty. For each element x of B, we compute the length w_x of the primitive string U such that U^5 occurs at position $i-2|U|$ of T and $\lfloor \log |U| \rfloor = x$ as follows. For each $x \in B$, we query for the primitive squares that occur at position j and have length in $[2^x\,..\,2^{x+1})$ using 4.2. For each result of these queries, i.e., for each of the at most two returned

primitive squares UU for each $x \in B$, we perform a periodic extension query that takes $\mathcal{O}(1)$ time to check if this square extends to an occurrence of U^5 at position $i - 2|U|$ in T; this will give us a unique U due to Theorem 4.6, yielding $w_x = |U|$. Next, for each $x \in B$, we test whether the vertical period of $A[i-2h\,..\,i+3h)[j\,..\,j+3w_x)$ is h using 2-period queries for strings $V^y_j[i-2h\,..\,i+3h)$ and $V^y_{j+3w_x-1-2^y}[i-2h\,..\,i+3h)$, where $y = \lfloor \log(3w_x) \rfloor$, and checking whether both returned periods are equal to h.[4] Then, $\mathsf{width}(R)$ is equal to the largest w_x for which this test is successful, if there is one, due to Theorem 4.10.

Finally, we postprocess the obtained $\mathcal{O}(\log n)$ elements in $\mathcal{O}(\log n)$ time to remove any elements that are dominated, thus obtaining the sought skyline of primitive 2D strings.

Computation of Dominated 2D Strings. Let the skyline 2D strings be $S_1, \ldots, S_\ell$, sorted in the order of decreasing width (and thus increasing height); the set of these 2D strings is henceforth denoted by $\mathcal{S}$.

We say that a 2D string U (strictly) dominates a 2D string V if and only if $g(U)$ (strictly) dominates $g(V)$. We also refer to chains and antichains of 2D strings based on this dominance relation. By Theorem 3.9, for each element of $\mathcal{S}$, the primitive 2D strings that span it horizontally (resp. vertically) form a chain.

For a set of strings $\mathcal{X}$, let us denote $g(\mathcal{X}) := \{g(X) : X \in \mathcal{X}\}$. Our aim is to compute an $\mathcal{O}(\log n)$-size set $\mathcal{G} \subset [1\,..\,\log n]^2$ such that $\mathcal{G} \supseteq g(\mathsf{assign}(i,j))$. We initialise $\mathcal{G}$ as $\mathcal{S} \cup \{(a,b) : (a,b)$ is dominated by $(a',b) \in \mathcal{S}$ or $(a,b') \in \mathcal{S}\}$. Let $\mathcal{R}$ be the set of primitive 2D strings that are dominated by at least one element of $\mathcal{S}$ and span either horizontally or vertically each of the elements of $\mathcal{S}$ that dominate them. Our main aim is to compute $g(\mathcal{R}) := \{g(R) : R \in \mathcal{R}\}$ and then set $\mathcal{G} = \mathcal{G} \cup g(\mathcal{R})$, which by Theorem 3.8 is a superset of $g(\mathsf{assign}(i,j))$. In the end, for each $(a,b) \in \mathcal{G}$ we infer a primitive 2D string R with $g(R) = (a,b)$ and check whether $R^{5,5}$ has an occurrence anchored at position (i,j) using a constant number of 2-period queries for fragments of strings from $\mathcal{H} \cup \mathcal{V}$. We next show how to compute $g(R)$.

Fact 4.11. For any k, a 2D string that horizontally (resp. vertically) spans $S_k \in \mathcal{S}$ also spans all S_t with $t > k$ (resp. $t < k$) that dominate it.

Consider a primitive 2D string $R \in \mathcal{R}$ and let the elements of $\mathcal{S}$ that dominate it be $S_k, \ldots, S_t$. Then, R horizontally spans a (possibly empty) suffix of $S_k, \ldots, S_t$, R vertically spans a (possibly empty) prefix of $S_k, \ldots, S_t$, and the sum of the lengths of the suffix and the prefix is at least $t-k+1$. In what follows, we will process the elements of $\mathcal{S}$ in the order of decreasing width. We will aim to compute $g(R)$ for each $R \in \mathcal{R}$:

- When processing the widest element of $\mathcal{S}$ that strictly dominates R (and is spanned horizontally by it),

  or, if such a point does not exist,

- when processing the thinnest element of $\mathcal{S}$ that strictly dominates R (and is spanned vertically by it).

We first process S_1, reporting (the dimensions of) the following 2D strings:

- The primitive 2D strings that vertically span S_1 and are not dominated by any other element of $\mathcal{S}$.

[4] The check for b^*, if it exists, is guaranteed to be successful.

- The primitive 2D strings that horizontally span S_1.

When processing S_k, for $k \in (1\,..\,\ell)$, we report (the dimensions of) the following 2D strings:

- The primitive 2D strings that vertically span S_k and are not dominated by S_{k+1}.
- The primitive 2D strings that horizontally span S_k and are not dominated by S_{k-1}.
- The primitive 2D strings that horizontally span S_k and vertically span S_{k-1}. We mark each 2D string that is reported by this procedure. We break the procedure whenever we encounter a 2D string that has already been marked.

Finally, we treat S_ℓ separately in $\mathcal{O}(\log n)$ time by computing (the dimensions of) all primitive 2D strings that span it either horizontally or vertically.

The above procedure reports $\mathcal{O}(\log n)$ (dimensions of) primitive 2D strings due to Theorem 3.11. As we will verify each of them separately in the end, we only need to show that we do not miss any of the 2D strings. By Theorem 3.10, missing any of them could only be due to the imposed stopping condition, that is, breaking upon encountering a marked 2D string. To that end, we utilize the following simple lemma, which, intuitively, states that horizontal and vertical spanning are transitive properties.

Lemma 4.12. Let X, Y, and Z be primitive 2D strings that satisfy $g(X) \leq g(Y) \leq g(Z)$. Then, the following two statements are equivalent:

(a) X horizontally (resp. vertically) spans Y and Y horizontally (resp. vertically) spans Z;

(b) X and Y horizontally (resp. vertically) span Z.

Proof. By symmetry, it suffices to prove the statement for horizontal spanning. Recall that Theorem 3.7 states that if S and T are two primitive 2D strings such that S spans T horizontally, then the horizontal period of $\mathsf{row}_{3\cdot\mathsf{height}(S)}(T^{2,2})$ is $\mathsf{width}(S)$. Since $\mathsf{width}(T)$ is also a horizontal period of $\mathsf{row}_{3\cdot\mathsf{height}(S)}(T^{2,2})$, $\mathsf{width}(S)$ divides $\mathsf{width}(T)$ and hence $\mathsf{row}_{3\cdot\mathsf{height}(S)}(T^{2,2}) = S^{3,2s}$ for some integer $s > 1$.

(a) $\Rightarrow$ (b): We have to show that X horizontally spans Z. Since $\mathsf{row}_{3\cdot\mathsf{height}(Y)}(Z^{2,2}) = Y^{3,2y}$ for some integer $y \geq 1$ and $\mathsf{row}_{3\cdot\mathsf{height}(X)}(Y^{2,2}) = X^{3,2x}$ for some integer $x \geq 1$, we have that $\mathsf{row}_{3\cdot\mathsf{height}(X)}(Z^{2,2}) = X^{3,2xy}$.

(b) $\Rightarrow$ (a): We have to show that X horizontally spans Y. We have that $\mathsf{row}_{3\cdot\mathsf{height}(Y)}(Z^{2,2}) = Y^{3,2y}$ for some integer $y \geq 1$ and $\mathsf{row}_{3\cdot\mathsf{height}(X)}(Z^{2,2}) = X^{3,2x}$ for some integer $x \geq 1$. Further, since $\mathsf{width}(X) \leq \mathsf{width}(Y)$, we have that $x \geq y$. First, observe that by the periodicity lemma (Theorem 2.1), $3\cdot\mathsf{height}(X) < 2\cdot\mathsf{height}(Y)$, else Y would not be primitive. Then, $\mathsf{row}_{3\cdot\mathsf{height}(X)}(Y^{2,2})$, which consists of the first $2 \cdot \mathsf{width}(Y)$ columns of $X^{3,2x}$ has both $\mathsf{width}(X)$ and $\mathsf{width}(Y)$ as its horizontal periods. By combining another application of the periodicity lemma with the fact that X is primitive, $\mathsf{width}(X)$ must divide $\mathsf{width}(Y)$. This implies that $\mathsf{row}_{3\cdot\mathsf{height}(X)}(Y^{2,2}) = X^{3,2x/y}$, concluding the proof. □

Due to the above lemma, for any two elements X and Y in the chain of primitive 2D strings, where $g(X) \leq g(Y)$, and for some k, both X and Y horizontally span S_k and vertically span S_{k-1}, we have that X spans Y both vertically and horizontally. Conversely, each 2D string X' that spans Y both horizontally and vertically, must also horizontally span S_k and vertically span S_{k-1}. Thus, the intersection of:

- a chain of primitive 2D strings that horizontally span S_k and vertically span S_{k-1}, and

- a chain of primitive 2D strings that horizontally span $S_{k'}$ and vertically span $S_{k'-1}$

for $k \neq k'$ consists of the longest common suffix of these chains (in decreasing order with respect to the mapping $g(\cdot)$). Thus, we do not lose any primitive 2D strings due to our stopping condition, which enables us to avoid reporting the same 2D string multiple times (e.g., a primitive 2D string that spans all elements of $\mathcal{S}$ both horizontally and vertically).

We have thus reduced the problem in scope, in $\mathcal{O}(n^2 \log n)$ time over all positions, to the problem of being able to generate chains and intersections of chains online in the natural order from the bottom-right in $\mathcal{O}(1)$ worst-case time per chain element.

Generation of chains. It suffices to discuss how to generate the chain of primitive 2D strings that horizontally span a skyline 2D string S. Let $(\alpha, \beta) := g(S)$. We iterate over all primitive strings U such that U^3 occurs at position i of V_j^β and $|U| \leq 2^\alpha$. Consider one such U and let $p := \lfloor \log |U| \rfloor$. We compute the period q of $H_i^{p+1}[j\,..\,j + 2 \cdot \mathsf{width}(S))$, which is at most $\mathsf{width}(S)$ using a 2-period query and conclude that $A[i\,..\,i + |U|)[j\,..\,j + q)$ horizontally spans S. Thus, we return $(p, \lfloor \log q \rfloor)$. Hence, processing a single value of $\lfloor \log |U| \rfloor$ takes $\mathcal{O}(1)$ time and is guaranteed to return a primitive 2D string that horizontally spans S. We efficiently retrieve all values of p one by one using the output of Theorem 4.8 applied to V_j^β with $k = 0$. This lemma returns a bitvector v whose a-th bit is set if and only if there is a sought U that satisfies $\lfloor \log |U| \rfloor = a$. It thus suffices to iterate over the set bits of this bitvector that are lower than α.

Generation of intersections of chains. We use tabulation, exploiting the following lemma.

Lemma 4.13. There exists a primitive 2D string R with $g(R) = (a, b)$ that spans S_k horizontally and S_{k-1} vertically if and only if the following conditions hold, where $(a_1, b_1) = g(S_k)$ and $(a_2, b_2) = g(S_{k-1})$:

- There is a primitive string W of length $w \in [2^b\,..\,2^{b+1})$ such that W^3 has an occurrence at position j of $H_i^{a_2+1}$ and the period of $H_i^{a+1}[j\,..\,j + |S_k|)$ is in $[2^b\,..\,2^{b+1})$;
- There is a primitive string U of length $h \in [2^a\,..\,2^{a+1})$ such that U^3 has an occurrence at position i of $V_j^{b_1+1}$, and the period of $V_j^{b+1}[i\,..\,i + |S_{k-1}|)$ is in $[2^a\,..\,2^{a+1})$.

Proof. The ($\Rightarrow$) direction is immediate.

($\Leftarrow$): The specified occurrence of W^3 implies that the first $3w$ columns of S_{k-1} have period w. This means that there is a primitive 2D string Y of width w that vertically spans S_{k-1}. In addition, the assumption that the period of $V_j^{b+1}[i\,..\,i + |S_{k-1}|)$ is in $[2^a\,..\,2^{a+1})$ implies that so is the height of Y. Then, due to Theorem 4.6, the specified occurrence of U^3 implies that $\mathsf{height}(Y) = h$. Hence, the string $A[i\,..\,i + h)[j\,..\,j + w)$ vertically spans S_{k-1}. By symmetry, the same string horizontally spans S_k. □

The above lemma means that, given

(1) The floor of the logarithm of the period of $V_j^b[i\,..\,i + |S_{k-1}|)$ for all b,

(2) The floor of the logarithm of the period of $H_i^a[j\,..\,j + |S_k|)$ for all a, and

(3) The bitvector computed by an application of Theorem 4.8 for $k = 0$ for position j of $H_i^{a_2+1}$ and position i of $V_j^{b_1+1}$,

one can compute the set $\{g(R) : R \text{ spans } S_k \text{ horizontally and } S_{k-1} \text{ vertically}\}$ in $\mathcal{O}(\log^2 n)$ time by iterating over all possible pairs (a, b). We next show that the information specified in (1) and (2) can be represented in $\mathcal{O}(\log n)$ bits and can be computed efficiently (in a batched computation), as is already the case for (3). This will allow us to use tabulation. In what follows, we focus on the representation and the computation of (1) as (2) is symmetric.

We encode the information on how the floor of the logarithm of the period of $V_j^b[i\,..\,i + |S_{k-1}|)$ changes when we iterate over $b \in [0\,..\,\lfloor\log(\mathsf{height}(S_{k-1}))\rfloor]$. After increasing b, this value either stays the same or increases. As the logarithm of the period always belongs to $[0\,..\,\lfloor\log(\mathsf{height}(S_{k-1}))\rfloor]$, this suggests the following natural encoding. We write down a sequence $0^{x_0}10^{x_1}10^{x_2}\ldots0^{x_\ell-1}1$, where $\ell = \lfloor\log(\mathsf{height}(S_{k-1}))\rfloor$. The bth 1 (counting from 0), corresponds to $V_j^b[i\,..\,i + |S_{k-1}|)$, and the total number of 0s before it is equal to the floor of the logarithm of its period. This represents the required information and consists of $\mathcal{O}(\log n)$ bits.

Further, the specified representation can be computed, for a fixed j, over all choices of i and k in $\mathcal{O}(n \log n)$ total time. This can be done by processing all runs in V_j^b in a manner analogous to that in the proof of Theorem 4.7. We build a forest of all runs whose length is at least three times larger than their period (instead of five), and then, for each position, for each of the $\mathcal{O}(\log n)$ considered root-to-leaf paths, we build the specified representation in time linear in the number of special pointers on that path. The result follows by the fact that the total number of special pointers traversed for each position is $\mathcal{O}(\log n)$. We need to retrieve the bitvector constructed for a leaf and apply a bitmask to keep only the bits that correspond to runs whose length is at least $|S_{k-1}|$ and period is at most $|S_{k-1}|$.

A direct attempt at using tabulation would be to preprocess the answer for every combination of the bitvectors encoding the information described in (1), (2), and (3). This is however $c \cdot \log n$ bits, for $c = 6$, so we cannot simply precompute the answer for each such input. Instead, we apply the standard approach of partitioning each of the bitvector into a constant number of smaller bitvectors, so that we have smaller instances of the same problem for which we can naively precompute the answers. This is described in more detail below.

Let $\alpha > 0$ be a constant to be chosen later. The bitvectors are partitioned into fragments corresponding to a range of at most $\alpha \cdot \log n$ values of a and b, respectively. This partition is chosen to additionally ensure that the floor of the logarithm increases by at most $\alpha \cdot \log n$ when we iterate over those values. Thus, the whole description of what is the floor of the logarithm consists of $2\alpha \cdot \log n$ bits, the initial value of a or b, and the floor of the logarithm of the period for this initial value, which takes $2 \log\log n$ additional bits, and we only need to create $2/\alpha$ fragments from each bitvector. Then, we precompute the elements of the sought chain for every pair of possible fragments. By adjusting the sufficiently small constant $\alpha > 0$ this takes $o(n)$ time. For each possible bitvector (describing the situation for the values of a or b), we preprocess its partition into $2/\alpha = \mathcal{O}(1)$ fragments. This takes only $\mathcal{O}(n^2 \log n)$ time and space, as the bitvectors are of length $2 \log n$. Then, when given the bitvectors, we first retrieve their partition into $2/\alpha = \mathcal{O}(1)$ fragments each. Next, we iterate over all $(2/\alpha)^2 = \mathcal{O}(1)$ combinations of fragments describing a range of values of a and b, retrieve the precomputed answer, and return the union of their precomputed chains. The chains are stored in the natural order, and we return $(2/\alpha)^2 = \mathcal{O}(1)$ pointers to their lists, which allows us to stop as soon as we see an element that has been already reported while spending only constant time per reported element.

Verification. Now, for each pair (a, b) in $\mathcal{G}$, we aim to compute the primitive 2D string R, if one exists, such that $R^{5,5}$ has an occurrence anchored at position (i, j). We compute the height of h, if it exists, as follows.

- Case I: $2h > 3 \cdot 2^a$. We have that $3h \geq 4 \cdot 2^a$. Then, since $2h < 2 \cdot 2^{a+1}$, h must be the period of $V_j^{b+1}[i \,..\, i + 2^{a+2})$, which can be computed in $\mathcal{O}(1)$ time using a 2-period query.
- Case II: $2h \leq 3 \cdot 2^a$. h is the period of $V_j^{b+1}[i \,..\, i + 3 \cdot 2^a)$, which can be computed in $\mathcal{O}(1)$ time using a 2-period query.

We perform both of the above 2-period queries and try to extend the obtained periods using periodic extension queries in V_j^{b+1}; at most one of these extensions will be successful due to Theorem 4.6. We symmetrically compute a candidate width w or conclude that no such R exists. Finally, assuming that we have obtained a candidate height h and a candidate width w, we compute R or conclude that none exists by checking, using a constant number of 2-period queries on strings from $\mathcal{H} \cup \mathcal{V}$, whether $A[i - 2h \,..\, i + 3h)[j - 2w \,..\, j + 3w)$ has vertical period h and horizontal period w.

4.4.2 Computation of Thick Quartics from Sets $\mathbf{assign}(i, j)$

We process all found occurrences of $R^{5,5}$ as in [25, Section 5]. While the complexity claimed in previous work was $\mathcal{O}(n^2 \log^2 n)$, the underlying method can be used to process a set S of found occurrences of $R^{5,5}$ in $\mathcal{O}(|S|)$ time to calculate, for every $x \geq 5$, the largest $y \geq 5$ such that an occurrence of $R^{x,y}$ can be composed of the given occurrences, assuming that the pairs in the set are lexicographically sorted. Overall, this sums up to $\mathcal{O}(n^2 \log n)$. Note that while sorting a single set with radix sort would take $\mathcal{O}(|S| + n)$ time, we can sort all the sets together, paying the additive $\mathcal{O}(n)$ only once. We briefly explain how to process a set S in the claimed time complexity. The reader can find a detailed description in [25, Section 5]. We stress that this is not meant to be a new result, but just a summary of a known method given for completeness.

First, we partition the pairs (i, j) into subsets corresponding to distinct values of $(i \bmod \mathsf{height}(R), j \bmod \mathsf{width}(R))$. Any fragment of the form $R^{x,y}$ with $x, y \geq 5$ is composed of occurrences of $R^{5,5}$ from the same subset. Thus, each subset can be processed separately. Further, we note that partitioning into subsets can be done in $\mathcal{O}(|S|)$ time using a $\mathsf{height}(R) \times \mathsf{width}(R)$ array of initially empty lists. The array is allocated only once at the very beginning of the whole procedure and reused for every S (after having processed each S, we clear the array). Further, we can assume that the pairs in every subset are sorted.

We explain how to process a subset $S' \subseteq S$. The problem now reduces to the following. We are given a set of white cells in a grid $[1 \,..\, n]^2$ and want to report all pairs (h, w) such that there exists an $h \times w$ rectangle consisting of white cells. Further, the white cells are lexicographically sorted. This allows us to process them row-by-row while maintaining an array $H[1 \,..\, n]$. When processing row r, we want every $H[c]$ to be equal to the height of the tower of white cells above position (r, c), that is, the largest k such that $(r, c), (r, c-1), \ldots, (r, c-k+1)$ are white cells. This can be maintained in time proportional to the number of white cells in the previous and the current row. Next, we process maximal fragments $H[i \,..\, j]$ consisting of non-zero entries. The goal is to process such a fragment in $\mathcal{O}(j - i + 1)$ time. We first find, for every $k \in [i \,..\, j]$, the nearest value smaller than $H[k]$ to the left and to the right of k in $H[i \,..\, j]$, denoted by $H[\mathsf{left}[k]]$ and $H[\mathsf{right}[k]]$, respectively. This can be done with two sweeps over $[i \,..\, j]$ (left-to-right and right-to-left) while maintaining a stack in $\mathcal{O}(j-i+1)$ time. Then, for $x = H[k]$, we set $y = \max\{y, \mathsf{right}[k] - \mathsf{left}[k] - 1\}$.

References

[1] Amihood Amir and Gary Benson. Efficient two-dimensional compressed matching. In Data Compression Conference, pages 279–288. IEEE Computer Society, 1992. doi:10.1109/DCC.1992.227453.

[2] Amihood Amir and Gary Benson. Two-dimensional periodicity in rectangular arrays. SIAM J. Comput., 27(1):90–106, 1998. doi:10.1137/S0097539795298321.

[3] Amihood Amir, Gary Benson, and Martin Farach. An alphabet independent approach to two-dimensional pattern matching. SIAM Journal on Computing, 23(2):313–323, 1994. doi:10.1137/S0097539792226321.

[4] Amihood Amir, Gary Benson, and Martin Farach. Optimal two-dimensional compressed matching. J. Algorithms, 24(2):354–379, 1997. doi:10.1006/JAGM.1997.0860.

[5] Amihood Amir, Gary Benson, and Martin Farach. Optimal parallel two dimensional text searching on a CREW PRAM. Inf. Comput., 144(1):1–17, 1998. doi:10.1006/INCO.1998.2705.

[6] Amihood Amir, Ayelet Butman, Moshe Lewenstein, and Ely Porat. Real two dimensional scaled matching. Algorithmica, 53(3):314–336, 2009. doi:10.1007/S00453-007-9021-X.

[7] Amihood Amir and Eran Chencinski. Faster two dimensional scaled matching. Algorithmica, 56(2):214–234, 2010. doi:10.1007/S00453-008-9173-3.

[8] Amihood Amir and Martin Farach. Two-dimensional dictionary matching. Inf. Process. Lett., 44(5):233–239, 1992. doi:10.1016/0020-0190(92)90206-B.

[9] Amihood Amir and Martin Farach. Efficient 2-dimensional approximate matching of half-rectangular figures. Inf. Comput., 118(1):1–11, 1995. doi:10.1006/INCO.1995.1047.

[10] Amihood Amir, Gad M. Landau, Shoshana Marcus, and Dina Sokol. Two-dimensional maximal repetitions. Theoretical Computer Science, 812:49–61, 2020. doi:10.1016/j.tcs.2019.07.006.

[11] Amihood Amir, Gad M. Landau, and Dina Sokol. Inplace 2d matching in compressed images. J. Algorithms, 49(2):240–261, 2003. doi:10.1016/S0196-6774(03)00088-9.

[12] A. Apostolico and V.E. Brimkov. Fibonacci arrays and their two-dimensional repetitions. Theoretical Computer Science, 237(1-2):263–273, 2000. doi:10.1016/S0304-3975(98)00182-0.

[13] Alberto Apostolico. Optimal parallel detection of squares in strings. Algorithmica, 8(4):285–319, 1992. doi:10.1007/BF01758848.

[14] Alberto Apostolico and Dany Breslauer. An optimal $\mathcal{O}(\log \log n)$-time parallel algorithm for detecting all squares in a string. SIAM J. Comput., 25(6):1318–1331, 1996. doi:10.1137/S0097539793260404.

[15] Alberto Apostolico and Valentin E. Brimkov. Optimal discovery of repetitions in 2D. Discrete Applied Mathematics, 151(1-3):5–20, 2005. doi:10.1016/j.dam.2005.02.019.

[16] Alberto Apostolico and Franco P. Preparata. Optimal off-line detection of repetitions in a string. Theor. Comput. Sci., 22:297–315, 1983. doi:10.1016/0304-3975(83)90109-3.

[17] Theodore P. Baker. A technique for extending rapid exact-match string matching to arrays of more than one dimension. SIAM Journal on Computing, 7(4):533–541, 1978. doi:10.1137/0207043.

[18] Hideo Bannai, Tomohiro I, Shunsuke Inenaga, Yuto Nakashima, Masayuki Takeda, and Kazuya Tsuruta. The "runs" theorem. SIAM Journal on Computing, 46(5):1501–1514, 2017. doi:10.1137/15M1011032.

[19] Hideo Bannai, Shunsuke Inenaga, and Dominik Köppl. Computing all distinct squares in linear time for integer alphabets. In CPM, pages 22:1–22:18, 2017. doi:10.4230/LIPICS.CPM.2017.22.

[20] Djamal Belazzougui, Dmitry Kosolobov, Simon J. Puglisi, and Rajeev Raman. Weighted ancestors in suffix trees revisited. In 32nd Annual Symposium on Combinatorial Pattern Matching, CPM 2021, pages 8:1–8:15, 2021. doi:10.4230/LIPICS.CPM.2021.8.

[21] Jean Berstel and Dominique Perrin. The origins of combinatorics on words. Eur. J. Comb., 28(3):996–1022, 2007. doi:10.1016/J.EJC.2005.07.019.

[22] Richard S. Bird. Two dimensional pattern matching. Inf. Process. Lett., 6(5):168–170, 1977. doi:10.1016/0020-0190(77)90017-5.

[23] S. Brlek and S. Li. On the number of squares in a finite word. arXiv, 2022. URL: http://arxiv.org/abs/2204.10204.

[24] Srecko Brlek and Shuo Li. On the number of distinct squares in finite sequences: Some old and new results. In Combinatorics on Words - 14th International Conference, WORDS 2023, pages 35–44, 2023. doi:10.1007/978-3-031-33180-0_3.

[25] P. Charalampopoulos, J. Radoszewski, W. Rytter, T. Waleń, and W. Zuba. The number of repetitions in 2D-strings. In 28th Annual European Symposium on Algorithms, ESA 2020, pages 1–18, 2020. doi:10.4230/LIPICS.ESA.2020.32.

[26] Ying Choi and Tak Wah Lam. Dynamic suffix tree and two-dimensional texts management. Inf. Process. Lett., 61(4):213–220, 1997. doi:10.1016/S0020-0190(97)00018-5.

[27] Raphaël Clifford, Allyx Fontaine, Tatiana Starikovskaya, and Hjalte Wedel Vildhøj. Dynamic and approximate pattern matching in 2D. In SPIRE, pages 133–144, 2016. doi:10.1007/978-3-319-46049-9_13.

[28] Maxime Crochemore. An optimal algorithm for computing the repetitions in a word. Information Processing Letters, 12(5):244–250, 1981. doi:10.1016/0020-0190(81)90024-7.

[29] Maxime Crochemore. An optimal algorithm for computing the repetitions in a word. Inf. Process. Lett., 12(5):244–250, 1981. doi:10.1016/0020-0190(81)90024-7.

[30] Maxime Crochemore, Leszek Gasieniec, Ramesh Hariharan, S. Muthukrishnan, and Wojciech Rytter. A constant time optimal parallel algorithm for two-dimensional pattern matching. SIAM J. Comput., 27(3):668–681, 1998. doi:10.1137/S0097539795280068.

[31] Maxime Crochemore, Leszek Gasieniec, Wojciech Plandowski, and Wojciech Rytter. Two-dimensional pattern matching in linear time and small space. In STACS 95, 12th Annual Symposium on Theoretical Aspects of Computer Science, pages 181–192, 1995. doi:10.1007/3-540-59042-0_72.

[32] Maxime Crochemore, Christophe Hancart, and Thierry Lecroq. Algorithms on strings. Cambridge University Press, 2007.

[33] Maxime Crochemore and Lucian Ilie. Maximal repetitions in strings. J. Comput. Syst. Sci., 74(5):796–807, 2008. doi:10.1016/j.jcss.2007.09.003.

[34] Maxime Crochemore, Lucian Ilie, and Liviu Tinta. The "runs" conjecture. Theor. Comput. Sci., 412(27):2931–2941, 2011. doi:10.1016/j.tcs.2010.06.019.

[35] Maxime Crochemore, Costas S. Iliopoulos, Marcin Kubica, Jakub Radoszewski, Wojciech Rytter, and Tomasz Walen. Extracting powers and periods in a word from its runs structure. Theor. Comput. Sci., 521:29–41, 2014. doi:10.1016/J.TCS.2013.11.018.

[36] Maxime Crochemore and Wojciech Rytter. Efficient parallel algorithms to test square-freeness and factorize strings. Inf. Process. Lett., 38(2):57–60, 1991. doi:10.1016/0020-0190(91)90223-5.

[37] Maxime Crochemore and Wojciech Rytter. Usefulness of the Karp-Miller-Rosenberg algorithm in parallel computations on strings and arrays. Theor. Comput. Sci., 88(1):59–82, 1991. doi:10.1016/0304-3975(91)90073-B.

[38] Maxime Crochemore and Wojciech Rytter. On linear-time alphabet-independent 2-dimensional pattern matching. In LATIN '95: Theoretical Informatics, pages 220–229, 1995. doi:10.1007/3-540-59175-3_91.

[39] Maxime Crochemore and Wojciech Rytter. Squares, cubes, and time-space efficient string searching. Algorithmica, 13(5):405–425, 1995. doi:10.1007/BF01190846.

[40] A. Deza, F. Franek, and A. Thierry. How many double squares can a string contain? Discrete Applied Mathematics, 180:52–69, 2015. doi:10.1016/J.DAM.2014.08.016.

[41] R. P. Dilworth. A decomposition theorem for partially ordered sets. Annals of Mathematics, 51(1):161–166, 1950. URL: http://www.jstor.org/stable/1969503.

[42] Jonas Ellert and Johannes Fischer. Linear Time Runs Over General Ordered Alphabets. 48th International Colloquium on Automata, Languages, and Programming (ICALP 2021), pages 63:1–63:16, 2021. doi:10.4230/LIPICS.ICALP.2021.63.

[43] Jonas Ellert, Pawel Gawrychowski, and Garance Gourdel. Optimal square detection over general alphabets. In Proceedings of the 2023 ACM-SIAM Symposium on Discrete Algorithms, SODA 2023, pages 5220–5242. SIAM, 2023. doi:10.1137/1.9781611977554.CH189.

[44] Martin Farach. Optimal suffix tree construction with large alphabets. In 38th Annual Symposium on Foundations of Computer Science, FOCS 1997, pages 137–143. IEEE Computer Society, 1997. doi:10.1109/SFCS.1997.646102.

[45] Nathan J. Fine and Herbert S. Wilf. Uniqueness theorems for periodic functions. Proceedings of the American Mathematical Society, 16(1):109–114, 1965. doi:10.2307/2034009.

[46] Aviezri S. Fraenkel and Jamie Simpson. How many squares can a string contain? Journal of Combinatorial Theory, Series A, 82(1):112–120, 1998. doi:10.1006/jcta.1997.2843.

[47] Zvi Galil and Kunsoo Park. Alphabet-independent two-dimensional witness computation. SIAM J. Comput., 25(5):907–935, 1996. doi:10.1137/S0097539792241941.

[48] P. Gawrychowski, S. Ghazawi, and Gad M. Landau. Lower bounds for the number of repetitions in 2D strings. In SPIRE 2021, pages 179–192, 2021. doi:10.1007/978-3-030-86692-1_15.

[49] Raffaele Giancarlo. A generalization of the suffix tree to square matrices, with applications. SIAM J. Comput., 24(3):520–562, 1995. doi:10.1137/S0097539792231982.

[50] Raffaele Giancarlo and Roberto Grossi. On the construction of classes of suffix trees for square matrices: Algorithms and applications. Inf. Comput., 130(2):151–182, 1996. doi:10.1006/INCO.1996.0087.

[51] Mathieu Giraud. Not so many runs in strings. In Language and Automata Theory and Applications, Second International Conference, LATA 2008, volume 5196, pages 232–239. Springer, 2008. doi:10.1007/978-3-540-88282-4_22.

[52] Mathieu Giraud. Asymptotic behavior of the numbers of runs and microruns. Inf. Comput., 207(11):1221–1228, 2009. doi:10.1016/j.ic.2009.02.007.

[53] Dan Gusfield. Algorithms on Strings, Trees, and Sequences - Computer Science and Computational Biology. Cambridge University Press, 1997.

[54] Dan Gusfield and Jens Stoye. Linear time algorithms for finding and representing all the tandem repeats in a string. J. Comput. Syst. Sci., 69(4):525–546, 2004. doi:10.1016/J.JCSS.2004.03.004.

[55] Jin-Ju Hong and Gen-Huey Chen. Efficient on-line repetition detection. Theor. Comput. Sci., 407(1-3):554–563, 2008. doi:10.1016/j.tcs.2008.08.038.

[56] Ramana M. Idury and Alejandro A. Schäffer. Multiple matching of rectangular patterns. Inf. Comput., 117(1):78–90, 1995. doi:10.1006/INCO.1995.1030.

[57] L. Ilie. A simple proof that a word of length n has at most 2n distinct squares. Journal of Combinatorial Theory, Series A, 112(1):163–164, 2005. doi:10.1016/J.JCTA.2005.01.006.

[58] L. Ilie. A note on the number of squares in a word. Theoretical Computer Science, 380(3):373–376, 2007. doi:10.1016/J.TCS.2007.03.025.

[59] Juha Kärkkäinen and Esko Ukkonen. Two- and higher-dimensional pattern matching in optimal expected time. SIAM J. Comput., 29(2):571–589, 1999. doi:10.1137/S0097539794275872.

[60] Donald E. Knuth, James H. Morris Jr., and Vaughan R. Pratt. Fast pattern matching in strings. SIAM Journal on Computing, 6(2):323–350, 1977. doi:10.1137/0206024.

[61] Tomasz Kociumaka. Efficient Data Structures for Internal Queries in Texts. PhD thesis, University of Warsaw, 2018. URL: https://mimuw.edu.pl/~kociumaka/files/phd.pdf.

[62] Tomasz Kociumaka, Jakub Radoszewski, Wojciech Rytter, and Tomasz Waleń. Internal pattern matching queries in a text and applications. In Proceedings of the Twenty-Sixth Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2015, pages 532–551. SIAM, 2015. doi:10.1137/1.9781611973730.36.

[63] Roman M. Kolpakov and Gregory Kucherov. Finding maximal repetitions in a word in linear time. In 40th Annual Symposium on Foundations of Computer Science, FOCS 1999, pages 596–604. IEEE Computer Society, 1999. doi:10.1109/SFFCS.1999.814634.

[64] Dmitry Kosolobov. Online square detection. CoRR, abs/1411.2022, 2014. arXiv:1411.2022.

[65] Dmitry Kosolobov. Online detection of repetitions with backtracking. In Combinatorial Pattern Matching - 26th Annual Symposium, CPM 2015, volume 9133, pages 295–306. Springer, 2015. doi:10.1007/978-3-319-19929-0_25.

[66] N. H. Lam. On the number of squares in a string. AdvOL-Report 2, 2013.

[67] Ho-fung Leung, Zeshan Peng, and Hing-Fung Ting. An efficient algorithm for online square detection. Theor. Comput. Sci., 363(1):69–75, 2006. doi:10.1016/J.TCS.2006.06.011.

[68] Michael G. Main and Richard J. Lorentz. An $\mathcal{O}(n \log n)$ algorithm for finding all repetitions in a string. J. Algorithms, 5(3):422–432, 1984. doi:10.1016/0196-6774(84)90021-X.

[69] Adam Marcus and Gábor Tardos. Excluded permutation matrices and the stanley-wilf conjecture. J. Comb. Theory, Ser. A, 107(1):153–160, 2004. doi:10.1016/J.JCTA.2004.04.002.

[70] Shoshana Neuburger and Dina Sokol. Succinct 2D dictionary matching. Algorithmica, 65(3):662–684, 2013. doi:10.1007/S00453-012-9615-9.

[71] Simon J. Puglisi, Jamie Simpson, and William F. Smyth. How many runs can a string contain? Theor. Comput. Sci., 401(1-3):165–171, 2008. doi:10.1016/J.TCS.2008.04.020.

[72] Azriel Rosenfeld and Avinash C. Kak. Digital Picture Processing: Volume 1 and 2. Computer Science and Applied Mathematics. Academic Press, Orlando, FL, 2 edition, 1982.

[73] Wojciech Rytter. The number of runs in a string: Improved analysis of the linear upper bound. In STACS 2006, 23rd Annual Symposium on Theoretical Aspects of Computer Science, pages 184–195, 2006. doi:10.1007/11672142_14.

[74] A. Thierry. A proof that a word of length n has less than $1.5n$ distinct squares. arXiv, 2020. URL: http://arxiv.org/abs/2001.02996.

[75] A. Thue. Über unendliche Zeichenreihen. Norske Vid. Selsk. Skr., I Mat.–Nat. Kl., Christiania, 7:1–22, 1906.